

Evaluation by Simulation

Fine-Tuning LLMs to Implement Code-Feedback

Master's Thesis
Computer Science

Author:	Andreas Lizenberger
Degree:	Master of Science
E-Mail:	andreas.lizenberger@student.uni-siegen.de
Handling Period:	08.10.25 - 08.04.26
Supervisor 1:	Prof. Dr. Roland Wismüller
Supervisor 2:	M. Ed. Sven Jacobs

Contents

1. Introduction	1
2. Background	5
2.1. Neural Networks	5
2.2. Sequential Data	5
2.3. Tokenizers and Embeddings	6
2.4. Recurrent Neural Networks	6
2.5. Long Short-Term Memory	7
2.6. RNN Encoder-Decoder	7
2.7. Attention	8
2.8. Transformers	8
2.9. Supervised Fine-Tuning	9
2.10. Reinforcement Learning	10
2.11. Group Relative Policy Optimization	10
2.12. LoRA: Low-Rank Adaptation	10
2.13. QLoRA	12
3. Methodology	15
3.1. Large Language Models	18
3.2. Zero-Shot Prompting	21
3.3. Supervised Fine-Tuning	23
3.4. GRPO Training	24
3.5. Evaluation	27
4. Results	28
4.1. Zero-Shot Prompting	28
4.2. Supervised Fine-Tuning	29
4.3. GRPO LLM-as-a-Judge	31
4.4. GRPO Training	33
5. Discussion	39
5.1. Zero-Shot Prompting	39
5.2. Supervised Fine-Tuning	41
5.3. GRPO Training	42
5.4. GRPO LLM-as-a-Judge	44
5.5. Implications	45
6. Related Work	48

7. Conclusion	50
8. Limitations	52
9. Future Work	55
References	58
A. Appendix	62

List of Figures

1.	Flow of data in Evaluation by Simulation.	2
2.	Normal distribution $\mathcal{N}(0, \sigma)$ and all 16 values $q_i \in [-1, 1]$ for NF4.	13
3.	Performance of Llama-3 for different sizes [32] [33] [31] [30] [29].	19
4.	Performance of Qwen2.5-Coder for different sizes [21].	20
5.	Rating based on difference in length, with $N = 100$	25
6.	Score for correctly placed reasoning tags.	35
7.	Score for correctly placed answer tags.	35
8.	Score for correct quantity of tags.	36
9.	Score for similar code lengths.	36
10.	Score given by ChatGPT as a judge.	37
11.	Total score over all reward functions.	37
12.	Unexpected crashes during training with GRPO.	38

Abstract

The integration of large language models (LLMs) as artificial teachers has brought new opportunities and challenges into the classroom. Computer science students can interact with these LLMs to obtain feedback on their program code. The automated evaluation of the quality of the code-feedback is important to support the learning efforts of the students and prevent the spread of misinformation. However, the automated evaluation of code-feedback is a challenging task with many factors. We suggest Evaluation by Simulation as a novel and pragmatic approach. Evaluation by Simulation leverages the coding capabilities of an LLM as artificial student to implement the code-feedback. By evaluating the quality of the generated code revision, we aim to indirectly measure the quality of the code-feedback. This requires a high degree of adherence to feedback from the artificial student. Adherence to feedback describes how closely the code revision reflects the code-feedback. Our goal is to maximize and measure the adherence to feedback in small-scale LLMs. We run 35 experimental setups with 4 small-scale LLMs (3B, 7B, 32B parameters) using zero-shot prompting, supervised fine-tuning, and GRPO training. We train reasoning models with GRPO in combination with an LLM-as-a-judge. We task the LLMs to implement code-feedback on an evaluation data set of 8 exercises, and classify the code revisions manually. Our results show that adherence to feedback is an inconsistent and unreliable capability of small-scale LLMs. Supervised fine-tuning achieves the best results in 2 of 35 experiments, with 4 of 8 code revisions showing adherence to feedback. In 27 out of 35 experiments, the LLMs fail to consistently implement the code-feedback and show a tendency to deviate towards either overcorrecting, or undercorrecting the program code, or failing the task outright. Our experiments with an LLM-as-a-judge highlight that the detection of overcorrecting behavior poses a challenge, with a recall of 55.81%.

1. Introduction

Large language models (LLMs) such as ChatGPT are powerful tools for text processing and text generation. For example, LLMs perform well on tasks such as classification, summarization, and question answering [5]. The impressive capabilities can be attributed to both the sheer volume of high quality data and the immense number of trainable parameters of the LLMs. The massive scale of these factors has enabled LLMs to demonstrate in-context learning. This means that LLMs can solve tasks that fall outside the expected scope of the training data [4]. The resulting versatility allows for the application of LLMs in various domains [5].

Throughout human history, the education of the future generations has always been a sensitive and important topic. Recent advances in artificial intelligence have enabled the integration of LLMs as artificial teachers into the education process. In education related to computer science, LLMs have been implemented to provide code-feedback (feedback) for programming exercises. The artificially generated feedback seems to be useful but fails to consistently point out specific errors in the students' program code. In some cases, the feedback contains hallucinations, referring to errors that are not actually present in the program code [22] [12]. Misinformation is a serious problem, because it negatively impacts the students' learning process. It is crucial to improve the quality of artificially generated feedback to equip students with accurate and reliable knowledge.

There is a diverse range of techniques and approaches that steer the behavior of LLMs. An application developer must choose which technologies to deploy. Among the choices for the LLM are propriety models like ChatGPT [1] by OpenAI, Gemini [48] by Google or open-source models like LLama [15] by Meta and Qwen [53] by Alibaba. These models are available with different variations that influence performance and pricing. Further adjustments can be made by tweaking the sampling parameters temperature, top-k, and top-p [37]. Prompt engineering can help steer the models towards better responses. Techniques include zero-shot prompting, few-shot prompting or chain-of-thought [43]. Fine-tuning is an effective method to increase the performance of models for downstream tasks [54]. And finally, we want to mention the growing number of technologies, RAG [26] and reasoning models [50] [16], that drastically improve the capabilities of LLMs to solve increasingly complex tasks.

A number of studies with LLMs as artificial teachers have been conducted [22] [12] [13] [36]. These studies consist of two phases. The data collection phase and the data evaluation phase. In the data collection phase, students engage with the artificial teacher and obtain feedback for their program code. In the data evaluation phase, experts assess the quality

of the generated feedback and report the results. It is important to conduct these studies to understand the capabilities and limits of LLMs as artificial teachers.

The problem is that these studies are not exploring a diverse range of LLM-related technologies. Each study explores the capabilities of a single artificial teacher. This can be attributed to two factors. First, the data collection phase is lengthy, taking multiple months to conclude. Second, the data evaluation phase relies on human evaluation. This is a time-intensive process as well. As a result, these studies operate within a relatively small scope and cannot keep up with the rapid advances in artificial intelligence. The findings of these studies can quickly become outdated and stale, especially with the release of newer models and technologies. In addition, these studies focus on highly specific downstream tasks and metrics, making it difficult to generalize, compare and reproduce the results. For example, the same artificial teacher for exercises on concurrency programming might struggle to provide the same quality of feedback for exercises on database queries.

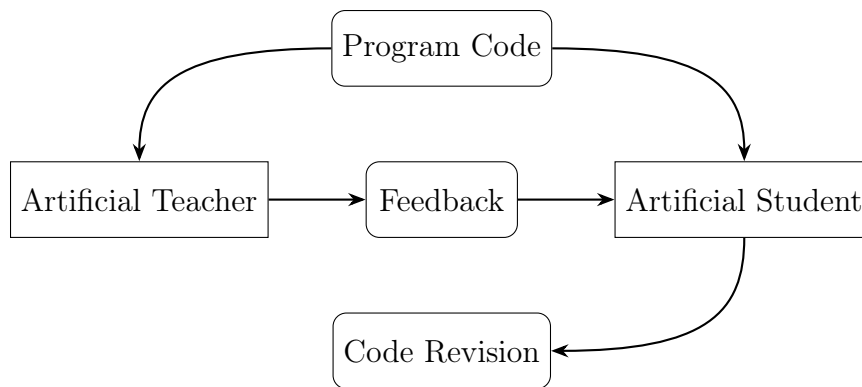


Figure 1: Flow of data in Evaluation by Simulation.

This motivates us to introduce Evaluation by Simulation (see Figure 1). Evaluation by Simulation conducts synthetic studies in an artificial classroom. An artificial classroom consists of an artificial teacher and artificial students. The artificial teacher generates feedback for the artificial students from given program code. The program code can be obtained through various means. For example, by introducing common programming errors into sample solutions, or repurposing available data from already existing studies. The artificial students generate code revisions from the given feedback and program code. The main idea is to utilize LLMs to simulate both the artificial teachers and the artificial students. This has the following benefits and applications:

1. Employ multiple artificial teachers: Each artificial teacher represents a set of LLM-related technologies. Evaluation by Simulation enables an application developer to configure and test out multiple artificial teachers. This makes it possible to explore the capabilities of multiple models, prompts, and settings.

2. Create student profiles: Students differ in their level of knowledge and understanding. Some students require feedback that is detailed and explanatory, while other students only need a small hint. Each artificial student can pose different needs and requirements towards the feedback.
3. Generate synthetic data sets: Evaluation by Simulation can produce synthetic data sets on a large scale within a short timeframe. Unlike real world studies, Evaluation by Simulation is neither restricted by the duration of the study nor the number of participants.
4. Automate evaluation: Evaluating the quality of the feedback directly requires expert knowledge in education and computer science. We can circumvent this requirement by evaluating the code revisions through an automated process instead. This can be achieved by simply running the code revisions through a compiler and unit tests. The quality of the feedback is measured by the impact of the feedback on the code revision.
5. Filter feedback: Evaluation by Simulation can be utilized as a filter to detect low quality feedback. Before the feedback is served to the real world student, the feedback must be served to the artificial student first. If the artificial student is unable to implement the feedback, then the feedback is discarded and re-generated. Only high quality feedback passes and is served to the real world student.

In order to fully realize Evaluation by Simulation, we must deploy suitable LLM-related technologies to implement an artificial student. Evaluation by Simulation relies heavily on the capabilities of LLMs to analyze the given program code, to interpret and follow the suggestions given by the feedback, and to generate a code revision that adheres closely to both program code and feedback. It is crucial that existing errors in the program code carry over to the code revision, unless explicitly addressed by the feedback. We refer to this set of capabilities as adherence to feedback. Adherence to feedback ensures that the quality of the code revision reflects the quality of the feedback.

Within the scope of this master’s thesis, we investigate small-scale LLMs with 3B and 7B parameters. Small-scale LLMs keep the hardware requirements to a minimum, enabling local deployment on a consumer-grade graphics card. This makes small-scale LLMs accessible to a wide audience of researchers and application developers. Using a small-scale LLM for the artificial student decreases maintenance and development costs and does not detract resources that are necessary for the artificial teacher. The smaller scale of these LLMs is beneficial for the exploration of various LLM-related technologies, such as fine-tuning, which may not be feasible with larger LLMs due to the substantial increase in hardware requirements. We argue that small-scale LLMs represent the lower bound for the capabilities of LLMs. Investigating the adherence to feedback of small-scale LLMs allows us to establish a baseline and limitations for further development.

We pose the research questions: When prompted to implement feedback into program code, to what extent can small-scale LLMs demonstrate adherence to feedback within the generated code revision?

We aim to answer the research question with a wide range of experimental setups. We investigate the influence of multiple model families, model sizes, prompt formats, and fine-tuning techniques on adherence to feedback. Notably, we employ supervised fine-tuning and GRPO training in an attempt to unlock and fully maximize the potential of small-scale LLMs. The GRPO training is combined with an LLM-as-a-judge approach to achieve reasoning models. Each experimental setup is evaluated manually. We prompt the LLMs to implement feedback into program code to obtain code revisions. We assign a category to each code revision to distinguish between adherence to feedback and deviation from feedback.

Our contribution is an empirical study that identifies that small-scale LLMs are frequently unable to implement the provided feedback when generating code revisions. Despite the deployment of various LLM-related technologies to alter this behavior, small-scale LLMs show a tendency towards either overcorrection, undercorrection or failure across 27 out of 35 experiments. The best 2 experiments, utilizing supervised fine-tuning, achieve adherence to feedback for 4 out of 8 exercises. This makes adherence to feedback an inconsistent and unreliable capability, limiting the practical application of small-scale LLMs as artificial students for Evaluation by Simulation. Additionally, LLM-as-a-judge shows a low detection rate for overcorrections in the code revisions, achieving a recall of 55.81%, confusing correct and valid code with code that adheres to feedback.

The software for our experimental setups can be found here:

<https://github.com/alpha-zero-one/llm-ai-student>

2. Background

In this background section, we explain the challenges and limitations of neural networks that have led to the development of the transformer architecture for LLMs. We discuss requirements for natural language processing and techniques to represent and extract information from text data. We outline supervised fine-tuning and GRPO training that we employ to train LLMs in this work. Finally, we explain parameter efficient fine-tuning as implemented by QLoRA.

2.1. Neural Networks

A neural network f is a parameterized function that maps an input x from $\mathbb{R}^k$ to $\mathbb{R}^d$. The idea is to construct f as a composition of deeply nested functions, also called layers. Each individual layer has a simple mathematical form and can have learnable parameters, also called weights [35]. A common nested function is a fully connected layer f_1 paired with a non-linear activation function g_1 :

$$f(x) = g_1(f_1(x)) = \sigma(Wx + b)$$

In a fully connected layer f_1 , we multiply the input $x \in \mathbb{R}^k$ with the weights $W \in \mathbb{R}^{d \times k}$ and add a bias $b \in \mathbb{R}^d$. The activation function g_1 can be a sigmoid function without any learnable parameters. A long chain m of these functions is called a deep neural network:

$$f(x) = g_m(f_m(\dots g_2(f_2(g_1(f_1(x)))) \dots))$$

The architecture of a neural network describes a specific composition of layers to construct the neural network. Initially, the weights of the neural network are randomly initialized. These weights can be trained, stored, and loaded from a previous state. Combining an architecture with trained weights forms a pre-trained model. The output of a neural network is called a prediction. The process of generating a prediction is called inference.

2.2. Sequential Data

Sequential data refers to sequences of variable lengths containing ordered elements. Unlike the data in a data set, sequential data must not be shuffled around, because sequential data rely on the order of their elements, especially the sequential relation between neighboring elements is important. For natural language, a sentence can lose its meaning when the positions of words are exchanged.

The length of sequential data is not predetermined. While it is possible to select a fixed maximum length, this results in a rigid and less scalable design. For example, it can be valid to determine a maximum sentence length, but that restricts sentences that are longer than this arbitrary sentence length. Sequential data pose a problem for neural networks. Since neural networks are fixed in input size, the processing of sequential data with dynamic lengths is not immediately supported [46].

2.3. Tokenizers and Embeddings

A fundamental task in natural language processing is the prediction of the next word from a given sequence of words. However, neural networks cannot operate on text directly. In order to feed a sentence into a neural network, we need to first convert each word into a number. This task is performed with a tokenizer. A tokenizer turns a sequence of words into a sequence of tokens. Each token corresponds to a single number. There are different types of tokenizer, such as word tokenizer and subword tokenizer. Subword tokenizers learn from a large corpus of text to identify common substrings and use these substrings to tokenize text, rather than tokenizing each word individually like word tokenizers [25].

The problem is that tokens alone do not hold much meaning. Rather than using tokens immediately, tokens are further processed by an embedding layer. An embedding layer maps a vector to a vector. The input vector is just a token represented as a vector in one-hot encoding. The output vector expresses the intensity of certain features in a vector space. The idea is to represent tokens with similar meaning close to each other in that vector space. For example, let us select German and English as our two features. We can plot all possible words in that vector space. The word “dog” is English, but not German. The word “bank” is a German and an English word. The word “katze” is German, but not English. The embedding in our example can help us determine whether the current sentence is written in German or in English. This restricts the next predicted word to that language. In practice, we do not enforce any semantic meaning in the vector space. Rather, the embedding is trained to model abstract features [34].

2.4. Recurrent Neural Networks

A recurrent neural network (RNN) is a special type of neural network that processes sequential data one element at a time. The input is the element at position n . The output is the prediction for the element at position $n+1$. In addition, RNNs recursively incorporate previous computations from the element at position $n-1$. This is achieved by adding these previous computations to the fully connected layer in the current computation [45].

Due to the recursive sequence of computations, RNNs become difficult to train with standard methods. Each step through the RNN decreases the training signal exponentially, also known as the vanishing gradient problem. Consequently, the weights in an RNN cannot be optimized with a steady rate. In practice, this results in an inability to learn from long sequences [45].

2.5. Long Short-Term Memory

Long Short-Term Memory (LSTM) is a response to the vanishing gradient problem. The idea is to perform simpler computations that do not exponentially decrease as the sequential data are processed for training. This is achieved by introducing short-term and long-term memories. These are 2 vectors that are only altered through additions and multiplications. LSTM is a special type of RNN, consisting of LSTM cells that follow a similar idea of processing each element in the input individually and adding computations from the previous iteration to the current iteration [18].

Each LSTM cell operates on 3 input signals and generates 2 output signals. The first input signal is the element at position n in the sequential data. The second input signal is the short-term memory from the previous iteration $n - 1$. The third input signal is the long-term memory from the previous iteration $n - 1$. LSTM cells run 3 operations to alter these short-term and long-term memories, resulting in the 2 output signals for the next iteration. The first operation is performed by the forget gate, which can return the long-term memory back to zero. The second operation is performed by the input gate, which can increase (or decrease) the long-term memory. The third operation is performed by the output gate, which determines the intensity of the short term memory derived from the long-term memory. The final prediction is achieved by applying a fully connected layer on the short-term memory of the last LSTM cell [18].

2.6. RNN Encoder-Decoder

RNN Encoder-Decoder is an implementation of LSTM to translate sentences. Translation functionally means predicting the next token in the target language. The idea is to split up the translation process into encoding and decoding steps [46] [7].

During the encoding steps, the encoder processes the sentence sequentially. The goal of the encoder is to accumulate a context vector that represents the entire sentence. For each token, the encoder updates the context vector. The final context vector is the input to the decoder.

During the decoding steps, the decoder sequentially predicts the next token based on the context vector. The goal of the decoder is to predict a special end-of-sentence token. For each previously predicted token, the decoder updates the context vector.

Both encoder and decoder consist of layered LSTM cells forming a grid. The short and long-term memory of an LSTM cell is the input of the LSTM cell that comes next in sequence. In addition, the short-term memory of an LSTM cell is the input of the LSTM cell in the layer above. The context vector at a position n represents the layered short and long-term memories stacked into a single vector.

2.7. Attention

While RNN Encoder-Decoder are suitable for the translation of short sentences, longer sentences pose a challenge. After encoding, the encoder compresses the entire sentence into a single dense context vector. During decoding, the decoder uses the context vector to track the progress of the translation as well. This dense compression can lose details over longer sequences, resulting in a decrease in performance [3].

Attention is a mechanism that aims to mitigate this problem. Attention suggests to retain and store all previous context vectors that are computed by the encoder. These context vectors of the encoder in addition to the current context vector of the decoder are utilized to inform the next token prediction [3].

The main idea is to compute the relevance of each context vector in the encoder with the current context vector in the decoder. Each context vector in the encoder is scaled by relevance. The attention vector is a weighted sum of these context vectors. The next token prediction utilizes both the current context vector and the attention vector to make the next token prediction [3]. This mechanism is called attention, because it allows the RNN to pay attention to specific and currently relevant tokens that were previously processed.

2.8. Transformers

The transformer architecture reconceptualizes Encoder-Decoder by introducing several monumental design changes. First, the transformer architecture breaks with the paradigm of sequentially processing tokens. Second, rather than encoding an entire sentence into a single dense context vector, transformers encode each token into a dense embedding. Third, transformers exploit self-attention to compute these dense embeddings [49].

Unlike RNNs, transformers do not process tokens sequentially. The major advantage is that transformers are highly parallelizable, since computations at position n do not rely on computations at position $n - 1$. In order to preserve the position of each token, positional encodings are added to the embeddings of each token. Positional encodings are vectors that can be computed with sine and cosine functions of varying frequencies.

In practice, transformers apply multiple self-attention layers in parallel to the positional embeddings of the token. This is called multi-head attention. Each attention head functionally is a more complex variant of an embedding layer. The positional embeddings are the input and dense embeddings are the output.

In concept, self-attention attempts to cluster relevant tokens together to determine a dense embedding for each token. Each token is assigned a value V , query Q , and key K . A value V is just a deeper embedding of the token. Each token poses a query Q : “How relevant are you to me?” All tokens must answer this query with a key K : “I am 20% relevant.” This relevance allows us to scale and sum all values together. The resulting dense embeddings represent the tokens after they have communicated with each other.

The encoder uses self-attention to compute dense embeddings for all tokens in the original input sequence. The decoder uses self-attention to compute dense embeddings for all tokens in the already generated output sequence. Finally, the decoder applies self-attention for all of these dense embeddings. The dense embeddings of the last token compress all relevant information for the next token prediction.

2.9. Supervised Fine-Tuning

Supervised fine-tuning is a training technique to adapt the weights of a neural network to a new task by utilizing a labeled data set. A label is the expected output for a sample in a data set. An optimizer trains the neural network over all samples to reproduce the associated labels. The neural network makes a prediction that is compared by a loss function to the expected label. A common loss function is the sum of square error loss. Gradient descent is an algorithm that can minimize this loss. The idea is to compute the gradient, scale the gradient with a small coefficient, and update the weights of the neural network by subtracting the scaled gradient. It is expensive to compute the gradient directly. Instead, backpropagation is implemented to compute partial derivatives by applying the chain rule backwards throughout the neural network [35].

When training a neural network on sequential data, the labels are implicitly given. For example, an LLM, that is build upon the transformer architecture, can learn to process text and predict new tokens. The prediction of the LLM is a vector of probabilities for

every single token. During supervised fine-tuning, the data set is constructed ad hoc. The LLM is trained on every possible subsequence of tokens. The token immediately following that subsequence is extracted as the label. The goal is to maximize the probability of that label.

2.10. Reinforcement Learning

Reinforcement learning is a training technique that can adapt the weights of a neural network by observing and evaluating the behavior of the neural network. Reinforcement learning does not rely on a labeled data set, instead training is formulated as a trial-and-error approach [14].

In reinforcement learning, an agent performs actions that influence the environment, resulting in a reward and a change of state. The agent utilizes a policy to select the next action. The agent observes the current state and receives the reward. The goal of the agent is to find a policy that maximizes the cumulative reward. Cumulative reward describes the sum of future rewards gained over a sequence of multiple actions [14].

In the context of LLMs, the LLM serves as the policy of the agent. The state is given by the current sequence of tokens. The action is described by the next generated token. The reward is not always provided directly, but can be received after generating a sequence of tokens [14].

2.11. Group Relative Policy Optimization

Group relative policy optimization (GRPO) is a reinforcement learning technique that can fine-tune LLMs. The idea is to prompt the LLM to generate multiple responses. These responses form a group. The optimization of the LLM occurs relative to the average reward of that group and is applied to all generated tokens within a response. The reward for each response is computed by using relatively simple reward functions. Responses that achieve a reward above the average are reinforced. Responses that achieve a reward below the average are discouraged. In addition, GRPO uses KL divergence to inhibit drastic changes to the weights of the LLM [44].

2.12. LoRA: Low-Rank Adaptation

In order to train a neural network, full fine-tuning is deployed. Full fine-tuning targets and trains all weights of the neural network. LLMs can have hundreds of billions of

parameters. For GPT3-175B, the GPU memory requirements can be as high as 1.2TB during full fine-tuning [19]. This poses a severe technical challenge, such as building and maintaining a system with specialized hardware components that support work in parallel, making fine-tuning inaccessible and expensive. We would need over 15 NVIDIA® A100 GPU, each with 80GB VRAM, to enable full fine-tuning.

LoRA is a technique to reduce the GPU memory requirements dramatically. Instead of targeting all weights, LoRA introduces additional weights in form of significantly smaller decomposition matrices of low rank. LoRA injects those low rank decomposition matrices into the largest fully connected layers, such as the query and value projection matrices of the LLM. All of the original weights of the LLM itself are frozen and not changed at all. LoRA approximates the change of the original weights during the training by updating the weights of the decomposition matrices instead [19]. Example:

$$f(x) = Wx + b$$

In a fully connected layer f , we multiply the input $x \in \mathbb{R}^k$ with the weights $W \in \mathbb{R}^{d \times k}$ and add a bias $b \in \mathbb{R}^d$. LoRA introduces the decomposition matrices $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ of rank r :

$$f(x) = Wx + BAx + b$$

The Adam optimizer allocates for each parameter three additional values in memory, one for the gradient vector and two for the moment vectors [24]. That means, when training LLMs with Adam, the GPU memory requirements grow with the number of trainable parameters multiplied by three. By freezing the original weights and only training the LoRA weights of B and A , we can avoid allocating any additional values for the significantly bigger weight matrix W . Since the size of B and A scales with the rank r , the VRAM requirements can be lowered by choosing r significantly smaller than d and k . The authors report for $r = 4$ that the GPU memory requirements for training GPT3-175B drop to 350GB, 29.17% of the original 1.2TB, and the number of trainable parameters drops to 0.02% compared to full fine-tuning [19].

Reducing the number of trainable parameters benefits checkpointing and storing the final LLM. Every checkpoint contains the trainable parameters of the LLM at fixed intervals. Full fine-tuning stores a full-sized LLM with every checkpoint. This is slow and consumes large quantities of disk space. Using LoRA, we only need to store the weights of the decomposition matrices. We can recover the training results by loading the original model and adding the trained LoRA weights dynamically. This approach is especially useful when the same LLM is trained for multiple downstream tasks. Each downstream task can be solved by loading the respective trained LoRA weights [19].

Method	WikiSQL	MNLI-m
Full Fine-Tuning	73.8%	89.5%
Fine-Tuning with LoRA	74.0%	91.6%

Table 1: Comparison of accuracy for both methods for GPT3-175B [19].

Despite training only a fraction of the weights, fine-tuning with LoRA has the potential to reach performance on par with full fine-tuning (see Table 1). In addition, when choosing a LoRA rank r equal to 4 or higher, training with LoRA results in LLMs that deliver nearly indistinguishable performance. In practice, this means that the rank is not an important hyperparameter to obtain more performance [19].

2.13. QLoRA

QLoRA is a collection of techniques designed to dramatically reduce the GPU memory requirements needed for loading and fine-tuning LLMs. For example, Guanaco-65B can be trained with 41GB VRAM using QLoRA, down from 780GB VRAM when using full fine-tuning. QLoRA is based on LoRA and adds the following advancements: 1) 4-bit data type NF4, 2) Double Quantization, 3) Paged Optimizers [10].

Type	Length	GPU Memory
FP32	32 bits	$4\text{byte} \cdot 7 \cdot 10^9 = 28\text{GB}$
FP16	16 bits	$2\text{B} \cdot 7 \cdot 10^9 = 14\text{GB}$
int8	8 bits	$1\text{B} \cdot 7 \cdot 10^9 = 7\text{GB}$
NF4	4 bits	$0.5\text{B} \cdot 7 \cdot 10^9 = 3.5\text{GB}$

Table 2: GPU memory requirements to load a 7B parameter LLM with various data types.

QLoRA introduces NF4, a 4-bit data type that aims to efficiently load the parameters of an LLM while keeping the quantization error to a minimum. In order to initialize an LLM for training or inference, we must allocate GPU memory for the parameters of the model first. There are data types of varying lengths that can store these parameters. NF4 is the shortest among the supported data types (see Table 2). By choosing NF4, we can maximize the reduction of the GPU memory requirements. This enables inference and training of larger LLMs on lower-end hardware or scaling the training to larger LLMs without upgrading the existing hardware.

NF4 promises information-theoretically optimal storage of the weights of the LLM. First, the authors assume that weights in a trained neural network follow a normal distribution $\mathcal{N}(0, \sigma)$. By subdividing the weights into blocks of size 64, each block of weights has

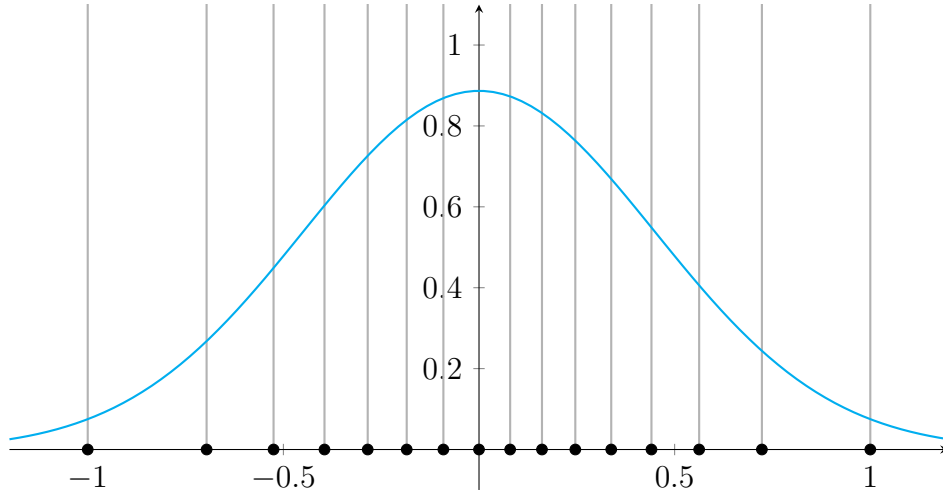


Figure 2: Normal distribution $\mathcal{N}(0, \sigma)$ and all 16 values $q_i \in [-1, 1]$ for NF4.

its own maximum absolute value c_i . These values c_i are the quantization constants of the blocks and are stored separately. By normalizing the blocks by dividing with c_i , the absolute maximum value of each block of weights is 1. This results in blocks of weights in the interval $[-1, 1]$ that follow a normal distribution. Second, the authors compute the 16 values q_i that are representable with NF4 (see Figure 2). The idea is that a normal distribution $\mathcal{N}(0, \sigma)$ has optimal values q_i that serve as bins of equal chance. When picking random numbers following $\mathcal{N}(0, \sigma)$ then the random numbers are equally distributed across all bins. The authors select $\mathcal{N}(0, 1)$ to compute those bins and then scale to the interval $[-1, 1]$ to derive q_i . Finally, each weight in the block of weights is quantized to one of those 16 optimal values q_i . As a result, we can expect that each block of weights distributes its weights across all values of NF4 in an equal manner. This ensures that the quantization error is kept minimal, while still representing the absolute maximum values of each block as accurately as possible. Note: In order to represent the number 0 exactly, the authors actually compute 7 negative bins, 8 positive bins and leave 1 bin for zero [10].

Double Quantization refers to the quantization of the quantization constants c_i . The quantization constants are stored as 32-bit values. Since each block of weights consists of 64 values, we would need to store an additional 0.5 bits per parameter. The authors suggest using double quantization with blocks of size 256. That means each block of quantization constants is subject to quantization, resulting in a second level of quantization constants. The authors subtract the mean of each block from c_i and scale to 8-bit length. This results in an additional 0.125 bits per parameter for the first quantization instead of the previous 0.5 bits per parameter. The second level of quantization constants are 32-bit long and span over $64 \cdot 256$ parameters, resulting in an additional ≥ 0.002 bits per parameter for the double quantization [10].

QLoRA utilizes Unified Memory of CUDA® by NVIDIA® to implement paged optimizers. Unified Memory is a feature that allows the system to treat portions of memory with all systems processing units (GPU and CPU) natively and concurrently. This enables GPU Memory Oversubscription. That means a GPU can operate on data that would not normally fit entirely into GPU memory. QLoRA uses this feature to manage optimizer states and call this technique paged optimizers. The authors describe their implementation of paged optimizers functionally similar to memory paging between RAM and disk. Paged optimizers increases the stability of the system by avoiding crashes due to unexpected memory spikes [10] [39].

3. Methodology

Our goal is to simulate an artificial student who can generate code revisions with adherence to feedback. The artificial student is provided with feedback and program code. The program code is written in the programming language python and bundled into a single python file. The feedback is articulated and presented in a natural language. The feedback gives hints and tips how to handle issues and errors that are present in the program code. Adherence to feedback means that a) the code revision must be based on the program code, b) the code revision must contain all changes that are mentioned in the feedback, and c) the code revision must not include any changes that are not explicitly mentioned in the feedback. Example:

Program Code:

```
1. def hello_world()  
2.     prnt("Hello World!")
```

Feedback:

The function definition is missing a colon after the parentheses.
The “print” function is misspelled as “prnt”.

Code Revision:

```
1. def hello_world():  
2.     print("Hello World!")
```

Note: The code revision adheres to feedback. The missing closing parenthesis in line 2 is not mentioned in the feedback and therefore must not be corrected in the code revision.

We utilize a large language model (LLM) to simulate the artificial student. LLMs are neural networks based on the transformer architecture and are capable of text generation, such as natural language generation and code generation. The size of the LLM and the quality of the training data set influence the capabilities of the LLM. Prompt refers to the input for the LLM and response refers to the generated output of the LLM. Example:

Prompt:

You are an expert in python programming.
You create a code revision by implementing the feedback.
<program code> ... </program code>
<feedback> ... </feedback>

Response:

I will happily do that! Here is the code revision in python:

```
<code revision> ... </code revision>
```

The problem is that the generated responses can vary drastically. LLMs predict the next words based on the previously seen sequence of words. Sometimes this works well, and sometimes this causes problems. The LLM can forget to generate a code revision, fail to follow the expected output format for the automated extraction of the code revision, return a copy of the program code without changing anything, solve issues that are not mentioned in the feedback, generate gibberish, or switch between different languages. The list of possible misbehavior is long!

We want to measure how frequently the LLM generates a code revision with adherence to feedback. We construct a data set of entry-level python exercises. Each exercise describes erroneous program code and matching feedback. The feedback does not necessarily cover all errors that are present in the program code. For each exercise, we prompt the LLM to generate a code revision, we extract the code revision from the generated response, and evaluate for adherence to feedback.

Unfortunately, the evaluation of the code revisions cannot be done in a fully automated manner. This is due to the fact that there is no reliable tool to evaluate the generated responses for our definition of adherence to feedback. Metrics for the evaluation of the coding capabilities of LLMs, such as MBPP, avoid the problem by running unit tests on the generated code. This approach assumes that the generated code must be syntactically correct and runnable. This assumption does not hold true for our case. The program code may contain errors and other defects. Depending on the feedback, some of these errors must be carried over to the code revision. This motivates us to conduct a manual evaluation without relying on automated tools.

All of our experiments are conducted in German, that includes the prompts, feedback, and code comments. This is because our proprietary training data set is based on interactions between German students and an artificial teacher. For the purpose of better readability, we choose to present the English translation within the scope of this work. Our experimental setups are as follows:

1. Zero-Shot Prompting
2. Supervised Fine-Tuning
3. GRPO Training

Zero-shot prompting is a prompting technique where the prompts must not contain any examples. We use zero-shot prompting for the following purposes. First, we want to measure the out-of-the-box capabilities of LLMs. We are interested in finding out how well the language and coding capabilities of LLMs translate to our downstream task of implementing feedback. Second, we suggest two prompting approaches to construct prompts that steer the LLM to follow our definition of adherence to feedback. We retain the same two prompts across all experiments to keep the results comparable. Third, we use zero-shot prompting to establish a baseline that we can compare the results of the other experimental setups to. This helps us to determine how fine-tuning impacts the capabilities of the LLMs.

Supervised fine-tuning is a training technique that teaches LLMs to generate responses to prompts based on a data set of prompt-response pairs. This approach is employed to improve the capabilities of LLMs to solve more specific downstream tasks. We collect real world data of interactions between human beginner programmers with an artificial teacher who provides code-feedback. The idea is that the LLM learns how to implement feedback in a manner similar to beginner programmers. We re-use the two prompts from zero-shot prompting to construct two data sets of prompt-response pairs. We train the LLMs for a set of six learning rates in logarithmic steps between $[5e-4, 1e-6]$ and evaluate each of the resulting LLMs.

GRPO is a reinforcement learning technique that trains LLMs to generate responses that result in a high score. The idea is to generate multiple responses and measure the score with simple reward functions. However, the LLM-as-a-judge approach suggests to utilize an additional LLM that serves as the judge by generating the scores for the responses instead. We utilize GRPO with the LLM-as-a-judge approach to train a reasoning model. A reasoning model is an LLM that thinks before generating an answer to the prompt. We hope that by introducing an additional thinking step the LLM improves on its capabilities to analyze the code and to determine how to best implement the feedback. Example:

Response:

<reasoning>

I must add a colon at the end of the first line. The print command is indeed incorrectly spelled. I must replace “prnt” with “print”. There is another error in line 2. The function call must end with a closing bracket, but the feedback does not mention this error. I must ignore the missing bracket.

</reasoning>

<answer> ... </answer>

The training of the LLMs is achieved on a local system with two NVIDIA® GeForce RTX™ 4090 GPUs and 48GB VRAM total. In case that training is not possible due to size constraints, we fall back on a system rented on runpod.io with a NVIDIA® A100 GPU and 80GB VRAM instead.

Setup	Loading	Training	Total
Full Fine-Tuning	$2 \cdot 7\text{GB}$	$3 \cdot 2 \cdot 7\text{GB}$	56GB
Fine-Tuning with QLoRA	$0.5 \cdot 7\text{GB}$	$\geq 0.05 \cdot 3 \cdot 2 \cdot 7\text{GB}$	$\geq 5.6\text{GB}$

Table 3: Estimated minimum GPU memory requirements for a 7B parameter LLM.

The challenge is to load the weights of the LLM into GPU memory, allocate memory for inference, and maintain enough memory for the optimizer states. A common approach to train a neural network is full fine-tuning. Full fine-tuning updates all weights during the training process. If we use 16-bit precision and a 7B parameter LLM, we can estimate that at least 56GB of GPU memory is required to load the LLM and setup the optimizer for training. 16-bit precision means that we need 2 bytes per parameter. In addition, the popular adam-optimizer needs to store 6 bytes per parameter. We recognize that our available VRAM is insufficient for full fine-tuning.

We utilize QLoRA to reduce the GPU memory requirements drastically and avoid full fine-tuning all of the weights. QLoRA offers to load the LLM in 4-bit precision and to train LoRA weights instead. LoRA weights make up a fraction of as little as 5% of the original weights of the LLM. The adam-optimizer for the LoRA weights requires three 16-bit values. This results in our estimate of at least 5.6GB GPU memory required for training with QLoRA. We set the LoRA rank to 8 and target all trainable dense layers for LoRA adaption.

3.1. Large Language Models

Huggingface is a hub for large language models and serves over two million LLMs of which more than 10% are specialized in text generation [20]. Among the highest downloads are LLMs from the Meta-Llama [15] and Qwen-Qwen [21] [53] family of models. At the time of our first experiments, the newest models are part of the Llama-3.2 and Qwen2.5 lines. The flagship models of both families have been shown to reach near state-of-the-art performance. We choose the following LLMs for our experiments:

- Llama-3.2-3B [33]
- Qwen2.5-Coder-7B [41]
- Qwen2.5-Coder-7B-Instruct [42]
- Qwen2.5-Coder-32B [40]

Our motivation is to perform Evaluation by Simulation. The idea is to use multiple LLMs to simulate the artificial students, and another LLM to simulate the artificial teacher. The artificial teacher generates multiple responses of feedback that the artificial students attempt to implement. That way we can indirectly evaluate the quality of the feedback by evaluating the quality of the implementation of the feedback.

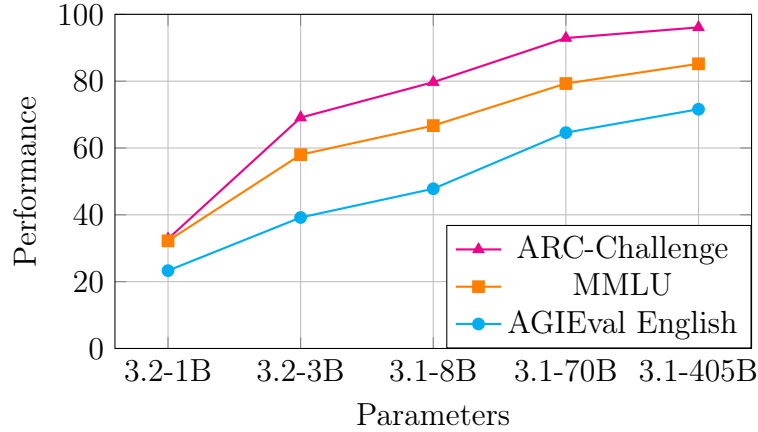


Figure 3: Performance of Llama-3 for different sizes [32] [33] [31] [30] [29].

The performance of LLMs within the same family scales with size (see Figures 3, 4). Llama-3.1-70B is 8.75 times bigger than Llama-3.1-8B and performs on average 23.53% better on general knowledge questions [31] [30]. Ideally, we choose the largest available models to obtain the best performance for the artificial students and the artificial teacher. The problem is that running multiple large LLMs comes with high hardware requirements. In order to enable Evaluation by Simulation, we decide to concentrate our experiments mostly on small-scale LLMs. Small-scale LLMs fit into our available GPU memory and are much simpler to run on our local system. For completeness, we also run the larger Qwen2.5-Coder-32B on an external system for a small number of experiments to see if the size difference has an immediate effect.

Another motivation to choose small-scale LLMs comes from a technical standpoint. Training and evaluating LLMs is a costly, time intensive, and error prone task. Libraries like huggingface-transformer [51] and unsloth [9] ease the difficulty but development remains a trial-and-error process. Loading, running, and debugging LLMs is a technical challenge that goes beyond just applying algorithm X to solve problem Y. Examples that work for one LLM might not work for another. Resources and documentation on possible solutions are scarce. We find that small-scale LLMs help with improving the development cycle because the smaller size reduces loading times, avoids unexpected out-of-memory issues and reduces the overall cost and complexity of development.

Llama-3.2 introduces a) vision models with 11B and 90B parameters that operate on text and images, b) smaller, high performing text generation LLMs with 1B and 3B

parameters. Llama-3.2-3B is the first model of the Llama-3 line below the 8B parameters mark and continues the trend of pruning and distillation to retain the capabilities of its predecessors Llama-3.1-8B and Llama-3.1-70B. Pruning is a technique to reduce the number of parameters of a neural network without impacting the performance by removing parameter that have low influence on the output. Distillation is a technique to train a model to reproduce the same output as another model. The idea is to shrink the number of parameters through pruning and realign the output through distillation [28].

Llama-3.2-3B is oriented towards dialog, text summarization, and writing assistance with support in multiple language such as English, German, French, Italian, Portuguese, Hindi, Spanish, and Thai. Llama-3.2-3B is not specialized in writing program code. Meta does not report on the coding capabilities in the official model card [33]. We use Llama-3.2-3B to simulate an artificial student with the capabilities of a beginner programmer. We hope that unclear or low quality feedback results in matching erroneous code revisions. Ideally, this means that the provided feedback of the artificial teacher must be more thorough and detailed in order allow the artificial student to generate a correct code revision. This is useful to separate feedback with unclear instructions from feedback with specific and detailed instructions.

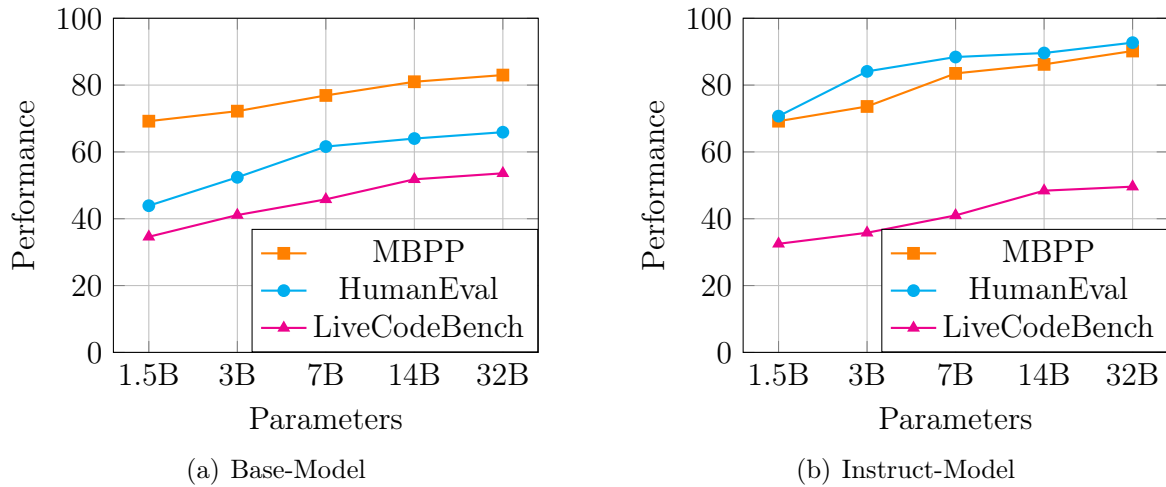


Figure 4: Performance of Qwen2.5-Coder for different sizes [21].

Qwen2.5-Coder is an LLM specifically designed for code generation. Qwen2.5-Coder has been trained by fine-tuning Qwen2.5 on 5.5 trillion tokens that span a variety of coding related documents, such as source code, mixed text-and-code data, and mathematical documents [21]. We hope to leverage the coding capabilities of Qwen2.5-Coder to realize an artificial student that follows the feedback very closely. We want to steer the model towards strictness. This can help us to identify feedback that misses crucial information to generate the correct code revision.

3.2. Zero-Shot Prompting

LLMs generate responses based on a given prompt. The phrasing of the prompt has an influence over what the LLM is going to respond with and how the response is going to look like. For example, we can ask the LLM to write program code in a specific programming language. We can ask the LLM to avoid generating additional comments or unnecessary explanations. Unfortunately, we cannot be certain that the LLM understands our prompt, follows our prompt or has the capabilities to deliver a satisfying response. It is a process of trial and error to craft a prompt that allows the LLM to generate satisfying responses consistently. Initially, we attempt a prompt that is short and straight forward:

Initial Prompt:

You are a beginner programmer. Implement the following feedback.

```
<program code> [ program code ] </program code>
```

```
<feedback> [ feedback ] </feedback>
```

However, the immediate issue is that the LLM tends to overcorrect and generate additional remarks. We find that the LLMs correct errors that are not explicitly mentioned in the feedback. The LLM sometimes generates no code and instead talks about the program code or extends the feedback with more suggestions. We deem it insufficient to use a naive prompting approach. This motivated us to try out different prompting scenarios to emphasize the need for adherence to feedback and to avoid unnecessary remarks. Finally, we settle on the following two prompts for our experiments:

Prompt 1: Markdown

You are a novice and beginner in programming. Here is the task.

Write a revision of the incorrect program. The incorrect program has syntactic and semantic errors and is incomplete.

The incorrect program looks like this.

```
```python
```

```
 [program code]
```

```
```
```

The feedback is as follows. [feedback]

The goal of the revision is to retain the problems of the incorrect program.

The revision of the program is deliberately erroneous and only implements the feedback. Avoid comments and explanations in the revision.

Revision of the program:

```
```python
```

Prompt 1 focuses on retaining errors and mistakes that are present in the program code. We prompt the LLM to roleplay as a novice programmer with the intention to steer the LLM to reproduce the errors of the program code in the code revision. We repeatedly highlight that the program code and the revision intentionally contain programming errors. We surround the code blocks with markdown to steer the LLM to generate python code. Finally, we use a loose markdown fragment in the last line to prompt the model to immediately continue with the code revision without generating any additional remarks.

**Prompt 2: XML**

You are an expert in software errors and defects.

You demonstrate typical careless mistakes in `<program>`.

You wrote the following `<program>` code for the presentation.

`<program>`

[ program code ]

`</program>`

You ask the audience for `<feedback>`.

`<feedback>`

[ feedback ]

`</feedback>`

You implement the `<code>` exactly as it was given in `<feedback>`, and only as it was given in `<feedback>`.

All other errors are retained unchanged in `<code>` in order to receive further `<feedback>` from the audience.

`<code>`

Prompt 2 has a strong focus on the roleplaying aspect. The LLM is holding a presentation to demonstrate errors in the the program code. The LLM asks the audience for feedback for the code revision. This prompt is designed to inhibit the LLM from correcting more than necessary. The prompt indicates that this is an interactive communication where additional feedback is anticipated later in the interaction. The code blocks are surrounded by XML tags. We use an opening XML tag in the last line to suggest immediate code generation.

Both prompts seem to reduce the observed behavior that we have witnessed with the initial prompt. We have constructed and tuned both prompts on two exercises outside of our evaluation data set. Both prompts managed to produce results that show more promising adherence to feedback in our preliminary testing.

### 3.3. Supervised Fine-Tuning

Supervised fine-tuning allows us to train the LLM towards a specific downstream task by changing the weights of the LLM. The LLM learns from a data set of prompt-response pairs how to generate a response to a given prompt. In our case, we want the LLM to behave similar to a beginner programmer. We create a data set of such prompt-response pairs based on a proprietary data set of interactions between human students and an LLM that takes the role of the artificial teacher.

The proprietary data set contains program code that is handed in by university students for a graded course in python. The students must solve exercises every week and hand in their program code for grading. The program code is automatically graded and the students have infinite attempts. At any point, the students can consult the artificial teacher to obtain feedback. The artificial teacher is prompted to not reveal any solutions directly, but instead provides hints and tips to the students how to correct their program code, such as explaining programming mistakes and compiler errors.

We construct the training data sets for supervised fine-tuning in the following way. The prompt-response pairs require program code, feedback, and code revisions. We filter the proprietary data set for all instances where the students request feedback for their program code. This provides us with the necessary program code and feedback pairs. In order to obtain the corresponding code revision, we have to make an assumption: a student, who hands in program code and asks for feedback at timestamp  $t$ , produces the corresponding code revision at timestamp  $t + 1$ . We sort, filter, and concatenate the corresponding code revisions to the program code and feedback pairs. This gives us the training data set of tuples (program code, feedback, program revision). For each of the two prompts, we generate a separate training data set by filling in the placeholders.

We utilize the huggingface-transformer library [51] to train the following LLMs:

- Llama-3.2-3B
- Qwen2.5-Coder-7B
- Qwen2.5-Coder-32B<sup>1</sup>

Both the markdown data set and the XML data set contain at least 5000 samples with a total token length of 2.066.750 tokens. The samples consist on average of 413.35 tokens, with a minimum of 172 tokens, and a maximum of 647 tokens. We apply QLoRA with rank 16, alpha 16, targeting all linear layers and train with a dropout of 10%. The training is run with an adam optimizer, batch size of one and learning rates 5.0E-4, 1.0E-4, 5.0E-5, 1.0E-5, 5.0E-6, 1.0E-6 over both data sets with two epochs.

---

<sup>1</sup>We train the 32B parameter model externally on runpod.io to a limited extent, only utilizing the markdown data set with the learning rates 5.0E-5 and 1.0E-5.

### 3.4. GRPO Training

Reasoning models are LLMs that think before giving an answer. First, reasoning models must describe how to solve the task that is presented by the prompt. And then, reasoning models must generate an answer to the prompt. The idea is to allow reasoning models to bring up core concepts, mention relevant terms, summarize important details, and to construct a plan to solve the task. This is especially effective when the initial prompt is short and lacks detail. A reasoning model can utilize the previously generated thoughts to steer itself towards a complete and correct answer. Example:

**Response:**

<reasoning>

I must add a colon at the end of the first line. The print command is indeed incorrectly spelled. I must replace “prnt” with “print”. There is another error in line 2. The function call must end with a closing bracket, but the feedback does not mention this error. I must ignore the missing bracket.

</reasoning>

<answer> ... </answer>

We deploy GRPO training to turn LLMs into reasoning models. GRPO is a technique that utilizes reward functions to rate the responses of the LLM. For every sample in the training data set, the LLM generates multiple responses. The reward functions provide ratings for each of these responses. GRPO trains the LLM to generate responses that achieve high ratings.

GRPO training is not inherently limited to creating reasoning models. Therefore, the reasoning capabilities of the resulting reason models can vary. We choose the following reward functions to encourage the LLM to develop reasoning capabilities:

1. Is the reasoning correctly tagged with XML? (0.5 score)
2. Is the answer correctly tagged with XML? (0.5 score)
3. Are there no duplicate XML tags? (0.5 score)
4. Are program code and code revision similar in length? (1.0 score)
5. Is the code revision adhering to feedback? (4.0 score)

The first three reward functions rate the form of the response. This includes the correct usage of XML tags, such as the adequate placement and the correct quantity of the XML tags. This ensures that the LLM is penalized, when it fails to provide reasoning. In addition, this enables us to easily extract the code revision from the answer.

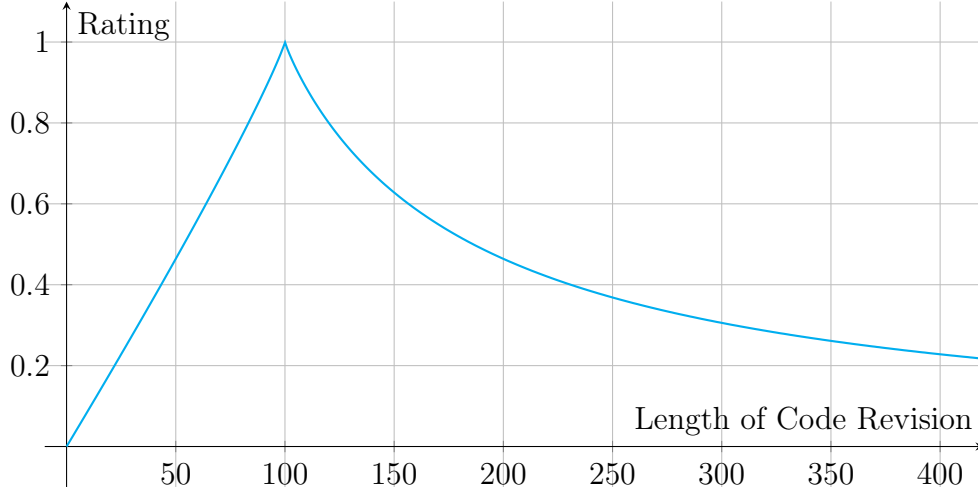


Figure 5: Rating based on difference in length, with  $N = 100$ .

The fourth reward function encourages the LLM to solve the task with minimal changes. The idea is to reward code revisions that are similar in character length relative to the initial program code. We lower the reward rapidly when the character lengths differ significantly. This ensures that the code revisions are not filled with unrelated text or bloated with comments. The following formula describes the reward for program code length  $N$  and code revision length  $X$ :

$$\text{score by length} = 1.0 - \left( \frac{\|N - X\|}{\max(N, X)} \right)^{0.9}$$

In order to realize the fifth reward function, we must evaluate the adherence to feedback of the code revisions. Since there is no simple mechanism to check for adherence to feedback, we deploy an LLM-as-a-judge approach. The LLM-as-a-judge approach utilizes an external LLM to rate the generated responses. We choose ChatGPT to serve as the judge. The task is to classify the code revisions into four categories: adherence to feedback, overcorrected, undercorrected, or failure (see Chapter 3.5 for more details). Adherence to feedback is rated with a score of 4.0, while the other categories yield no score.

We conduct separate experiments to construct an optimal judge. The goal is to find a judge that yields results close to our manual evaluation. We compile the results of the manual evaluation of the previous experiments into a data set with 224 samples. We prompt GPT-4.1 and GPT-4.1-mini with a concise prompt and a detailed prompt. The prompts contain the program code, feedback, and code revision. Additionally, we run the experiments with and without appending the code difference between the program code and the code revision. We find that the optimal judge is GPT-4.1-mini with the following prompt format:

**Prompt Judge: Concise with Code Diffence appended**

You validate commits. Assume `<old code>` and `<new code>` are exactly handed in like this. Consider the changes in `<code difference>`. Solve the tasks below step by step.

First: Find all changes made by `<new code>`. Create a list of all these changes briefly. Utilize all changes mentioned in `<code difference>`.

Second: The `<new code>` must implement the `<commit>`. In this context, what are undocumented changes?

Third: Consider the list from the first task. If there are any undocumented changes mark them. Create a table with columns (change, mentioned in commit?, undocumented?).

Fourth: Return the number that fits best.

Return 0: The `<commit>` message has no undocumented changes AND `<new code>` implemented all `<commit>` messages.

Return 1: There is at least one undocumented change.

Return 2: At least one implementation of a `<commit>` message is missing in `<new code>`.

Return 3: The `<commit>` message does not fit the implemented changes of `<new code>` at all!

`<old code>` [program code] `</old code>`

`<commit>` [feedback] `</commit>`

`<new code>` [code revision] `</new code>`

`<code difference>` [code difference] `</code difference>`

The prompt leans towards the validation of commit messages. The judge is guided step-by-step to identify all changes in the code revision. The judge must note down explicitly which changes are not mentioned by the feedback. The feedback is treated like a commit message. The idea is to increase the sensitivity of the judge towards deviations from the feedback and to pay attention to small details. We append the code difference to assist the judge in that regard. The judge must return a value that corresponds to exactly one of our four categories.

We utilize the unsloth [9] library to train the following LLM:

- Qwen2.5-Coder-7B-Instruct

We perform GRPO training on the markdown data set and XML data set that we have already constructed for supervised fine-tuning. The data sets consist of prompt-response pairs. Since GRPO is a reinforcement learning technique that relies on reward functions for training, we can disregard the existing responses. GRPO automatically generates 6 responses with the LLM during training. We apply QLoRA with rank 8, alpha 8, targeting all linear layers and train without dropout. The training is run with a paged adam optimizer, batch size of 6 and learning rate set to  $5e-6$  over both data sets with a single epoch.

### 3.5. Evaluation

Adherence to feedback means that a) the code revision must be based on the program code, b) the code revision must contain all changes that are mentioned in the feedback, and c) the code revision must not include any changes that are not explicitly mentioned in the feedback. We derive 4 categories to report on the quality of the code revisions:

- Adherence to feedback: The code revision satisfies conditions a, b, and c.
- Overcorrected: The code revision only fails to satisfy condition b.
- Undercorrected: The code revision only fails to satisfy condition c.
- Failure: The code revision fails to satisfy two or more conditions.

These categories allow us to determine the behavior of the LLM. Overcorrected code revisions indicate that the LLM ignores the restraints of the feedback. The LLM can solve the task but favors correctness over adherence to feedback. Undercorrected code revisions indicate that the LLM is not capable of solving the task within the constraints of the feedback. For example, the LLM generates a code revision that is just a copy of the program code. We consider code revisions that show any further deviations as failure.

We construct an evaluation data set of 8 entry-level python exercises in order to evaluate the LLMs in our experimental setups. The exercises are designed to be simple to implement and easy to evaluate by manual evaluation. The exercises consist of program code and feedback wrapped in the XML prompt and markdown prompt. The feedback addresses some of the errors in the provided program code. None of the exercises have been used for training. Each LLM in the experimental setups is prompted once to generate a code revision per exercise. We manually evaluate the code revisions by classifying the code revisions into four categories.



## 4. Results

We evaluate the performance of each LLM and report our findings in form of tables. The tables show the model, the configuration, and the evaluation on an evaluation data set that consists of eight samples. We let each LLM generate a single response per sample and extract the code revision. The evaluation classifies the code revisions into the following categories:

- The code revision (A) adheres to feedback.
- The code revision is (O) overcorrecting.
- The code revision is (U) undercorrecting.
- The code revision (F) fails.

### 4.1. Zero-Shot Prompting

We conduct zero-shot prompting for all models and list the results in Table 4. For Qwen2.5-Coder-32B, we only use prompt 1 that contains markdown.

Model	Prompt	A	O	U	F
Llama-3.2-3B	Markdown	2	1	5	0
	XML	1	2	5	0
Qwen2.5-Coder-7B	Markdown	0	6	2	0
	XML	1	5	2	0
Qwen2.5-Coder-7B-Instruct	Markdown	0	1	3	4
	XML	0	6	1	1
Qwen2.5-Coder-32B	Markdown	1	3	3	1

Table 4: Results for zero-shot prompting.

We observe a trend for each model. Llama-3.2-3B tends to undercorrect. Qwen2.5-Coder-7B shows a trend towards overcorrecting. Qwen2.5-Coder-7B-Instruct varies between the prompts. For markdown, the code revisions result in undercorrection and failure. For XML, there is a tendency to overcorrect. Qwen2.5-Coder-32B is split evenly across overcorrection and undercorrection.

The base models show similar performance independent of the utilized prompt. The instruction model shows a noticeable difference between XML and markdown. Across all configurations, we see that the LLMs tend to generate code revisions that are either overcorrected or undercorrected. The exception is Qwen2.5-Coder-7B-Instruct-Markdown with the most failed code revisions at a total of 4.

Llama-3.2-3B-Markdown performs best with 2 code revisions with adherence to feedback. The other configurations generate up to 1 code revision with adherence to feedback.

## 4.2. Supervised Fine-Tuning

We conduct supervised fine-tuning for Llama-3.2-3B (Tables 5, 6), Qwen2.5-Coder-7B (Tables 7, 8), and Qwen2.5-Coder-32B (Table 9). Each configuration of supervised fine-tuning takes between 6 and 8 hours to complete. Qwen2.5-Coder-32B is trained externally on runpod.io and takes 4 hours to complete instead.

Model	Prompt	Learning Rate	A	O	U	F
Llama-3.2-3B	Markdown	5.0E-4	2	2	2	2
		1.0E-4	0	1	3	4
		5.0E-5	0	0	5	3
		1.0E-5	1	1	4	2
		5.0E-6	1	1	4	2
		1.0E-6	1	1	6	0
Llama-3.2-3B	Markdown	No Training	2	1	5	0

Table 5: Supervised fine-tuning results for Llama-3.2-3B-Markdown.

Llama-3.2-3B-Markdown with learning rates between 1.0E-4 and 5.0E-6 shows a tendency towards undercorrection and failure. The smallest learning rate 1.0E-6 introduces no failure, but tends strongly towards undercorrection. The highest learning rate 5.0E-4 is split evenly among all categories. Except for learning rate 1.0E-6, all LLMs introduce failure. Learning rate 1.0E-4 results in a total of 4 failed code revisions.

Llama-3.2-3B-Markdown shows no improvement after fine-tuning. The best LLM with learning rate 5.0E-4 produces 2 code revisions with adherence to feedback. The other LLMs generate up to 1 code revision with adherence to feedback.

Model	Prompt	Learning Rate	A	O	U	F
Llama-3.2-3B	XML	5.0E-4	0	0	4	4
		1.0E-4	0	1	4	3
		5.0E-5	0	2	3	3
		1.0E-5	0	1	5	2
		5.0E-6	0	1	5	2
		1.0E-6	0	0	7	1
Llama-3.2-3B	XML	No Training	1	2	5	0

Table 6: Supervised fine-tuning results for Llama-3.2-3B-XML.

Llama-3.2-3B-XML tends towards undercorrection and failure. The lower learning rates 1.0E-5, 5.0E-6, 1.0E-6 lean towards undercorrection, and the higher learning rates 5.0E-4, 1.0E-4, 5.0E-5 are split evenly between undercorrection and failure.

Llama-3.2-3B-XML shows no improvement after fine-tuning. The LLMs do not generate any code revisions with adherence to feedback.

Model	Prompt	Learning Rate	A	O	U	F
Qwen2.5-Coder-7B	Markdown	5.0E-4	2	3	3	0
		1.0E-4	4	0	3	1
		5.0E-5	3	1	2	2
		1.0E-5	1	2	4	1
		5.0E-6	2	3	3	0
		1.0E-6	2	2	4	0
Qwen2.5-Coder-7B	Markdown	No Training	0	6	2	0

Table 7: Supervised fine-tuning results for Qwen2.5-Coder-7B-Markdown.

Qwen2.5-Coder-7B-Markdown is split evenly across adherence to feedback, overcorrected and undercorrected code revisions. There is a slight tendency towards undercorrection, where 5 out of 6 LLMs generate at least 3 undercorrected code revisions.

Qwen2.5-Coder-7B-Markdown achieves better performance after fine-tuning. All LLMs generate code revisions with adherence to feedback with 5 out of 6 LLMs generating at least 2 code revisions with adherence to feedback. The best performing LLM with learning rate 1.0E-4 generates a total of 4 code revisions with adherence to feedback.

Model	Prompt	Learning Rate	A	O	U	F
Qwen2.5-Coder-7B	XML	5.0E-4	1	0	1	6
		1.0E-4	2	2	3	1
		5.0E-5	0	2	5	1
		1.0E-5	1	1	4	2
		5.0E-6	2	2	4	0
		1.0E-6	4	0	3	1
Qwen2.5-Coder-7B	XML	No Training	1	5	2	0

Table 8: Supervised fine-tuning results for Qwen2.5-Coder-7B-XML.

Qwen2.5-Coder-7B-XML trends towards undercorrection with 5 out of 6 LLMs generating at least 3 undercorrected code revisions. The exception is the configuration with the highest learning rate 5.0E-4 with a tendency towards failure with 6 failed code revisions. The configuration with the lowest learning rate 1.0E-6 trends towards generating adherent and undercorrected code revisions.

Qwen2.5-Coder-7B-XML achieves better performance after fine-tuning. The learning rates 1.0E-4, 5.0E-6 perform better with 2 code revision with adherence to feedback. The learning rate 1.0E-6 performs best with 4 code revisions that adhere to feedback.

Model	Prompt	Learning Rate	A	O	U	F
Qwen2.5-Coder-32B	Markdown	5.0E-5	2	1	1	4
		1.0E-5	1	0	6	1
Qwen2.5-Coder-32B	Markdown	No Training	1	3	3	1

Table 9: Supervised fine-tuning results for Qwen2.5-Coder-32B-Markdown.

Qwen2.5-Coder-32B shows an improvement after fine-tuning. The learning rate 5.0E-5 generates 2 code revisions with adherence to feedback, but shows an increase to 4 failed code revisions. The learning rate 1.0E-5 matches the performance prior to fine-tuning, but shifts towards undercorrection.

### 4.3. GRPO LLM-as-a-Judge

We evaluate the performance of ChatGPT as a judge. The evaluation covers two models and two prompts, both with and without appending the code difference between the program code and the code revision. ChatGPT predicts a category for each sample in the evaluation data set. We compare the prediction with the actual category and report the sensitivity, specificity, recall on each category, and accuracy.

The evaluation data set is a compilation of 224 samples from the previous results on zero-shot prompting and supervised fine-tuning. The evaluation data set consists of 33 adherent, 43 overcorrected, 105 undercorrected and 43 failed code revisions. We note that ChatGPT has successfully returned a valid category for each of the samples across all configurations in our experiment.

Model	Prompt	Code Diff.	Sensitivity	Specificity
GPT-4.1	concise	missing	93.94%	84.82%
		appended	90.91%	84.82%
GPT-4.1-mini	detailed	missing	90.91%	81.15 %
		appended	93.94%	82.72%
	concise	missing	90.91%	80.10%
		appended	90.91%	88.48%
	detailed	missing	90.91%	85.86%
		appended	84.85%	87.96%

Table 10: Sensitivity and specificity across judges.

All configurations, except GPT-4.1-mini-detailed-appended, achieve a sensitivity above 90.91%. This indicates that the models correctly predict at least 30 out of 33 adherent samples as adherent. The specificity ranges between 80.10% and 88.48%. This indicates that in 191 cases where the code revision is deviating (overcorrected, undercorrected, or failed), the models predict between 153 to 169 cases as deviating.

The data suggests that model, prompt and code difference do not impact the sensitivity. Appending the code difference seems to improve the specificity. GPT-4.1 seems to perform better with a concise prompt, while GPT-4.1-mini achieves the best and worst result with a concise prompt. The specificity of GPT-4.1-mini is overall higher, but varies more than the specificity of GPT-4.1.

Model	Prompt	Code Diff.	Recall A	Recall O	Recall U	Recall F
GPT-4.1	concise	missing	93.94%	58.14%	73.33%	11.63%
		appended	90.91%	53.49%	78.10%	13.95%
GPT-4.1-mini	detailed	missing	90.91%	39.53%	79.05%	30.23%
		appended	93.94%	30.23%	91.43%	11.63%
	concise	missing	90.91%	39.53%	79.05%	9.30%
		appended	90.91%	55.81%	92.38%	23.26%
	detailed	missing	90.91%	44.19%	83.81%	58.14%
		appended	84.85%	51.16%	93.33%	11.63%

Table 11: Recall per evaluation category across judges.

Recall for adherence to feedback and sensitivity describe the same concept, therefore both columns have the same values ranging mostly between 90.91% and 93.94%. The recall on overcorrected code revisions fluctuates between 39.53% and 58.14%. The recall on undercorrected code revisions fluctuates between 73.33% and 93.33%. The recall on failed code revisions fluctuates between 9.30% and 58.14%.

The data suggests that ChatGPT agrees with our manual evaluation on adherence to feedback and undercorrected code revisions, but disagrees with our evaluation on overcorrected code revisions, and strongly disagrees on failed code revisions. This holds true across all models, prompts, with and without appended code differences.

Appending the code difference improves the detection of undercorrected code revisions. Appending the code difference decreases the detection of overcorrected code revisions for GPT-4.1, but increases the detection of overcorrected code revisions for GPT-4.1-mini. Failure detection seems to increase for the detailed prompt without providing the code difference.

We list the accuracy for two scenarios. Accuracy<sub>t</sub> describes the performance of ChatGPT as a judge for the purpose of fine-tuning a reasoning model. That means predicting an

Model	Prompt	Code Diff.	Accuracy <sub>t</sub>	Accuracy <sub>e</sub>
GPT-4.1	concise	missing	86.16%	61.61%
		appended	85.71%	62.95%
	detailed	missing	82.59%	63.84%
		appended	84.38%	64.73%
GPT-4.1-mini	concise	missing	81.70%	59.82%
		appended	88.84%	71.88%
	detailed	missing	86.61%	72.32%
		appended	87.50%	68.30%

Table 12: Training accuracy and evaluation accuracy across judges.

adherent sample as adherent, and a deviating sample as deviating. Accuracy<sub>e</sub> describes the performance of ChatGPT for the purpose of automated evaluation of responses. That means predicting the exact category.

Accuracy<sub>t</sub> ranges between 81.70% and 88.84%. This indicates that ChatGPT is a decent, but fallible judge. The training of a reasoning model benefits from a high accuracy. The misclassification of samples impacts the quality and duration of the training process negatively. Therefore, the best configuration is GPT-4.1-mini with a concise prompt and appended code difference.

Accuracy<sub>e</sub> ranges between 59.82% and 72.32%. ChatGPT is not suitable to determine the category of a code revision. This supports our claim that there is no adequate tool for automated evaluation of adherence to feedback. As a result, manual evaluation remains the only viable option to investigate the capabilities of an LLM for adherence to feedback.

#### 4.4. GRPO Training

We conduct GRPO training with an LLM-as-a-judge to obtain reasoning models. The judge is GPT-4.1-mini with a concise prompt and appended code difference. The reasoning models are derivatives of Qwen2.5-Coder-7B-Instruct. The training data set spans 5000 samples. After processing a batch of 1000 samples, we evaluate the performance of the reasoning models. Each batch of 1000 samples takes approximately 8 hours to compute, resulting in a total training duration of 40 hours per configuration.

Qwen2.5-Coder-7B-Instruct-Markdown shows a trend towards overcorrection. After 1000 samples, Qwen2.5-Coder-7B-Instruct-Markdown still generates a total of 4 failed code revisions. As the number of processed samples increases the number of failed code revisions decreases. The number of overcorrected samples increases with growing number of samples processed.

Model	Prompt	Samples	A	O	U	F
Qwen2.5-Coder-7B-Instruct	Markdown	1000	1	3	0	4
		2000	1	5	1	1
		3000	1	5	0	2
		4000	1	6	0	1
		5000	2	6	0	0
Qwen2.5-Coder-7B-Instruct	Markdown	No Training	0	1	3	4

Table 13: GRPO training results for Qwen2.5-Coder-7B-Instruct-Markdown.

Qwen2.5-Coder-7B-Instruct-Markdown achieves better performance after fine-tuning. All configurations generate at least 1 code revision with adherence to feedback. The best result is at 5000 samples processed with 2 code revisions with adherence to feedback.

Model	Prompt	Samples	A	O	U	F
Qwen2.5-Coder-7B-Instruct	XML	1000	0	6	1	1
		2000	0	4	1	3
		3000	0	6	1	1
		4000	0	6	1	1
		5000	0	4	1	3
Qwen2.5-Coder-7B-Instruct	XML	No Training	0	6	1	1

Table 14: GRPO training results for Qwen2.5-Coder-7B-Instruct-XML.

Qwen2.5-Coder-7B-Instruct-XML leans towards overcorrecting. At 2000 samples and at 5000 samples, there is a tendency towards failure. The results are similar across all processed samples.

Qwen2.5-Coder-7B-Instruct-XML shows no improvement after fine-tuning. Across all configurations, there is no code revision with adherence to feedback. The performance after fine-tuning with GRPO strongly resembles the performance of zero-shot prompting.

We utilize 5 reward functions during GRPO training. The average score per reward function is presented in Figures 6, 7, 8, 9, 10. The figures depict 25 points, where each point is an accumulation of 200 samples. GRPO generates 6 responses per sample. Therefore, each point represents the average score across 1200 responses.

In Figure 6, we see that the reasoning models for both prompts are capable of creating reasoning tags. The maximum score is 0.5, XML varies between 0.46 and 0.49, markdown varies between 0.48 and 0.5. XML starts out higher than markdown, but only shows marginal improvement. Markdown catches up to XML and slightly surpasses XML after processing 1200 samples and then stabilizes with no further improvement.

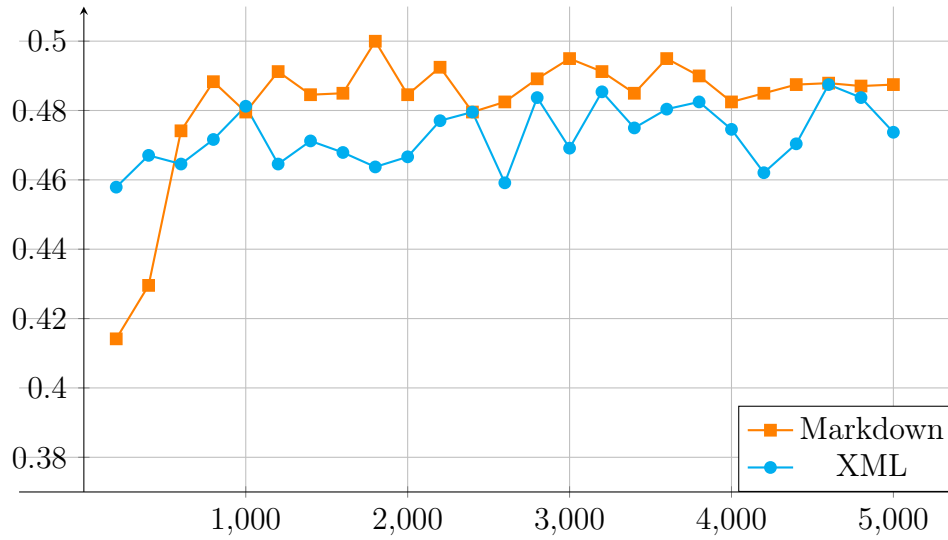


Figure 6: Score for correctly placed reasoning tags.

In Figure 7, we see that the reasoning models for both prompts are capable of creating answer tags. The maximum score is 0.5, XML varies between 0.44 and 0.48, markdown varies between 0.46 and 0.49. The reasoning models for both prompts improve the most after processing the first 1200 samples. For the remaining samples, the reasoning models fluctuate but show no further improvement. Markdown performs slightly better than XML. The score for the answer tags starts 10% lower than the score for the reasoning tags. After 1200 samples, the difference closes and narrows down to 2%.

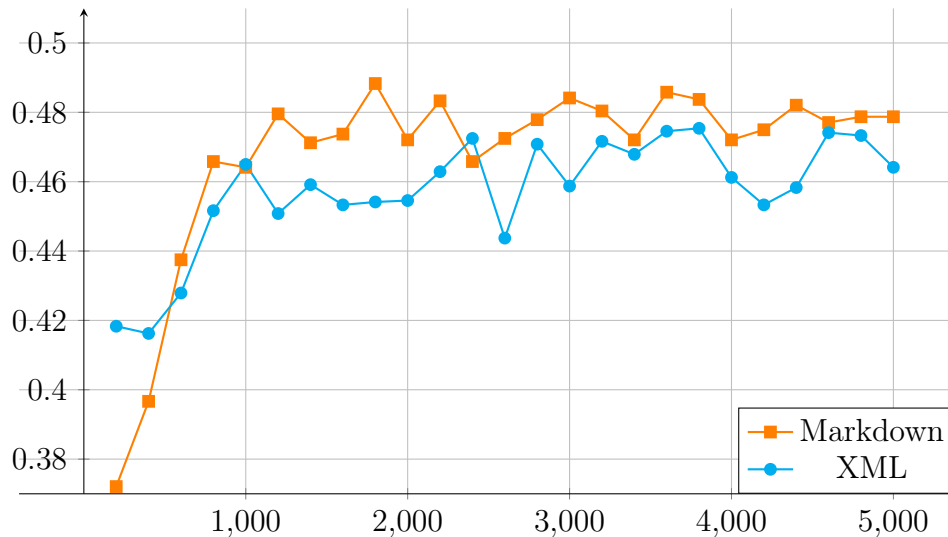


Figure 7: Score for correctly placed answer tags.

In Figure 8, we see the score for the usage of the correct quantity of opening and closing tags. Prior to 800 samples, the score for the correct quantity of tags starts out higher than the score of the answer tags. This shows that the reasoning models tend to create additional explanations after providing the code revision. We do not reward the reasoning



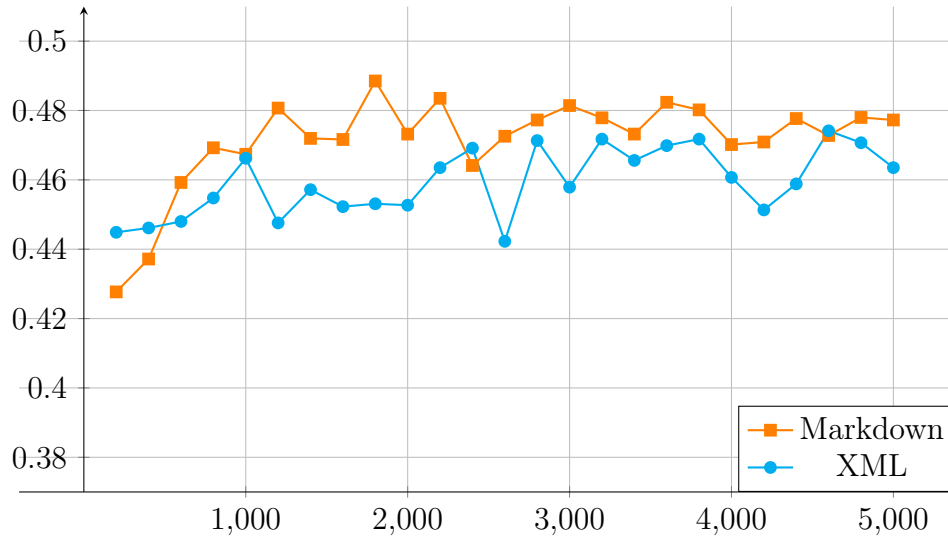


Figure 8: Score for correct quantity of tags.

models when the response ends with anything but the closing tag followed by empty spaces or line breaks. Past 800 samples, the graph is near identical to the previous graph for the score of the answer tags. The reasoning models learn to avoid creating additional explanations after the closing tag.

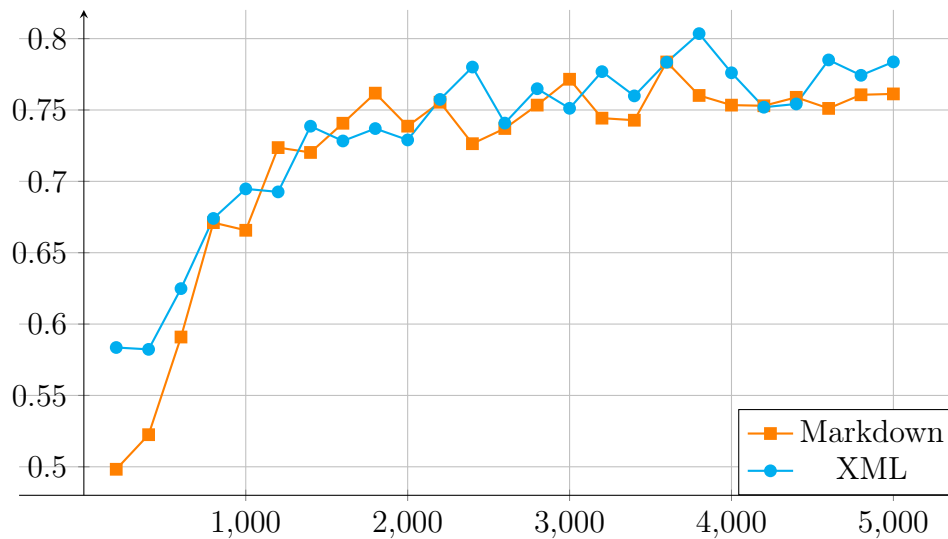


Figure 9: Score for similar code lengths.

In Figure 9, we see the score for the difference in lengths between the provided program code and the generated code revision. The maximum achievable score is 1.0. The score for markdown starts out as low as 0.5, which translates to 86% longer code revisions. This means that the code revisions contain more changes, more comments and additional explanations. After 4000 samples, both reasoning models seem to stabilize between a score of 0.75 and 0.8. That means that on average the generated code revisions are between 20% and 27% longer than the provided program code.

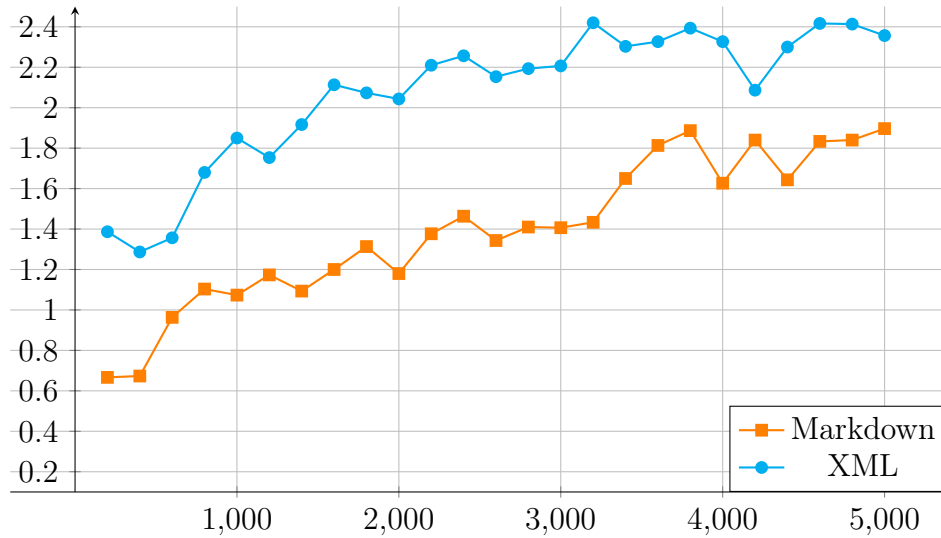


Figure 10: Score given by ChatGPT as a judge.

In Figure 10, we see the score that ChatGPT awards for adherence to feedback of the generated code revisions. The reasoning models for both prompts show linear improvement across all 5000 samples, but the reasoning models are unable to reach the maximum score of 4.0 points. XML reaches a score of 2.4 points, while markdown reaches a score of 1.9 points. Markdown is consistently below XML. The score of XML is on average higher by 0.6 points.

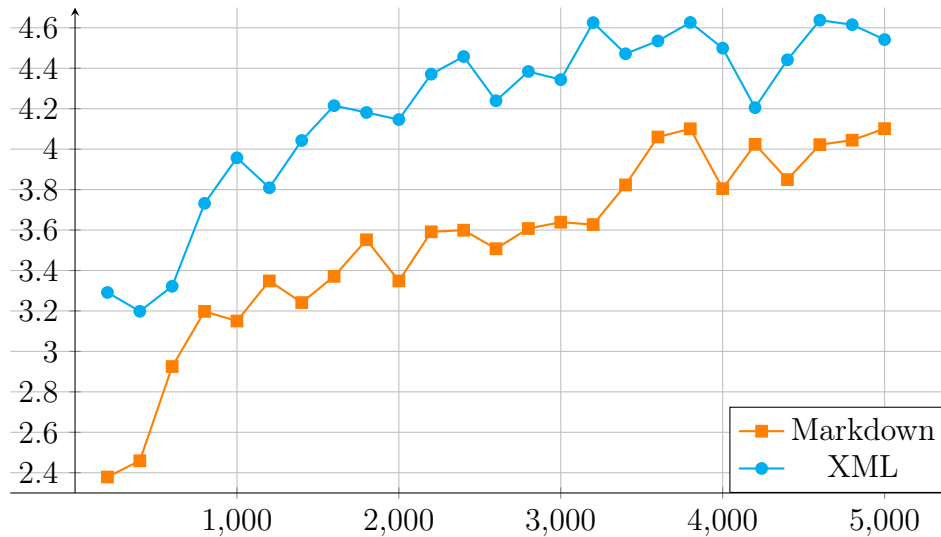


Figure 11: Total score over all reward functions.

In Figure 11, we see the total score over all reward functions. We note that the highest influence on the total score is the score given by the judge with 4.0 points. The shape of the graph strongly resembles the shape of the graph in Figure 10. All previous reward functions sum up to 2.5 points. This explains the steep initial improvement of the score for the first 1000 samples.

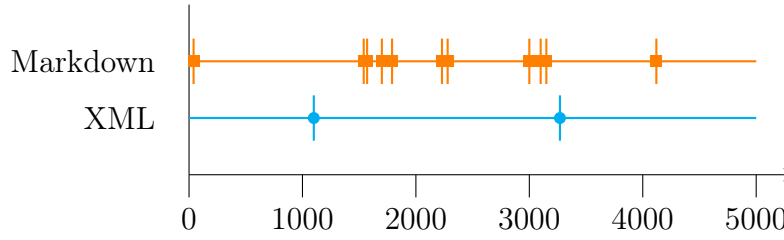


Figure 12: Unexpected crashes during training with GRPO.

The training process has been interrupted by unexpected software crashes. Figure 12 shows the occurrence of the crashes with respect to the number of samples processed. The training with the markdown prompt terminates 11 times prematurely, while the training with the XML prompt terminates 2 times prematurely. Unfortunately, the unsloth library does not report on the circumstance of the crash and the error stack trace is unavailable. We observe that ChatGPT failed to respond 10 times during the training with markdown, and 7 times during the training with XML, due to a temporarily network timeout. However, this network error is properly handled and not the direct cause for the software crashes.

## 5. Discussion

In this master’s thesis we investigate the research question: When prompted to implement feedback into program code, to what extent can small-scale LLMs demonstrate adherence to feedback within the generated code revision? In order to answer the research question, we discuss the results of our experimental setups in the following sections.

### 5.1. Zero-Shot Prompting

We discuss the results of the experiments with zero-shot prompting. Zero-shot prompting is necessary to establish the baseline behavior. This is crucial to gain insights into the fundamental tendencies of small-scale LLMs. We determine the extent to which adherence to feedback is already present in small-scale LLMs.

**Baseline performance:** Zero-shot prompting reveals a low degree of adherence to feedback for all of our experimental setups. LLama-3.2-3B-Markdown performs best with just 2 out of 8 code revisions that adhere to feedback. The other experimental setups generate either one or zero code revisions that adhere to feedback. This demonstrates that, out of the box, small-scale LLMs do not reliably generate code revisions with adherence to feedback.

**Prompting for adherence:** Both XML and markdown prompts emphasize the requirement to retain errors and only to implement the remarks stated in the feedback. Despite this explicit request all experimental setups show a deviation from adherence to feedback. Adherence to feedback is not a capability that can be taken for granted. Our utilized prompt scenarios have not been sufficient to enable adherence to feedback. We expect to see an even stronger deviation when utilizing prompts that lack this emphasis.

**Exercise complexity:** Our evaluation data set consists of relatively simple python exercises. The provided feedback addresses two issues at most and the program code spans less than 12 lines. We speculate that an increase in the complexity of the exercise description may negatively impact adherence to feedback. This would render small-scale LLMs unsuitable for detailed feedback and complex programming tasks.

**Model size:** It appears that the model size does not stand in direct correlation to the capability for adherence to feedback. Llama-3.2-3B performs best despite being the smallest model. Qwen2.5-Coder-32B is the largest model at 10 times the size. Yet, the size difference does not yield an immediate improvement in adherence to feedback. This is an indicator that adherence to feedback does not emerge at a small scale, regardless of the chosen size of the LLM.

**Tendency to deviate:** Our results show that 5 out of 7 experimental setups exhibit tendencies towards either overcorrection or undercorrection. Within those 5 experimental setups, at least 5 out of 8 generated code revisions are overcorrected or undercorrected. This points towards an underlying problem within the model families that interferes with adherence to feedback.

**Model specialization:** Llama-3.2-3B is a general-purpose LLM and not specialized for coding tasks. This can impose restrictions on the coding capabilities of the LLM. Llama-3.2-3B generates 5 undercorrected code revisions. We argue that this tendency towards undercorrection could be a consequence of the limited coding capabilities of Llama-3.2-3B, rather than a disregard for adherence to feedback.

**Language alignment:** Llama-3.2-3B is trained on multiple languages, including German. Since our prompts and feedback are phrased in German, this alignment in language can enable the LLM to put attention on certain keywords and identify relevant instructions. This may offer an explanation why Llama-3.2-3B generates the highest number of code revisions with adherence to feedback.

**Correctness over adherence:** Qwen2.5-Coder-7B is specialized for code generation. We observe a tendency towards overcorrection. This suggests that Qwen2.5-Coder-7B is generally capable of identifying and correcting the errors in the given program code. However, Qwen2.5-Coder-7B disregards the constraints specified in the feedback. This leads us to believe that Qwen2.5-Coder-7B prioritizes generating code revisions with valid programming syntax over adherence to feedback. This misalignment in priority can be explained with the composition of the training data. The training data consists of program code collected from GitHub. The training data is filtered for correctness with a compiler and a model for quality assessment, further removing samples with syntax errors. Consequently, Qwen2.5-Coder-7B fails to preserve existing errors in the program code, despite explicit instructions in the prompt.

**Format sensitivity:** Qwen2.5-Coder-7B-Instruct demonstrates different behavior for the two prompts. For XML, we obtain results that are similar to Qwen2.5-Coder-7B. For markdown, we see an increase in the failure category. This seems to suggest that the instruction-tuned model is sensitive to the format utilized in the prompt. Qwen2.5-Coder-7B-Instruct might associate different objectives with each format. XML is a strictly hierarchical format to store arbitrary data for machine processing and data transmission. Markdown is primarily used as a format for the presentation of text data, e.g. readme files. Utilizing XML seems to yield better coding capabilities, but neither format has an impact on adherence to feedback.

## 5.2. Supervised Fine-Tuning

We discuss the results of the experiments with supervised fine-tuning. Supervised fine-tuning trains and specializes the LLMs for our downstream task. We aim to modify the behavior by tuning the LLMs to imitate responses that implement feedback. This allows us to answer to what extent adherence to feedback represents a latent capability that needs to be unlocked through training.

**Varied impact:** Supervised fine-tuning influences the performance of the models to a varying degree across all configurations. For Llama-3.2-3B, we see a decrease in adherence to feedback. For Qwen2.5-Coder-7B and Qwen2.5-Coder-32B, we see an increase in adherence to feedback. This demonstrates that supervised fine-tuning can unlock adherence to feedback for certain models, but also can have the inverse effect for other models. This suggests that models have different aptitudes for the training of adherence to feedback. Thus, supervised fine-tuning is not a technique that can reliably ensure the emergence of adherence to feedback.

**Inconsistency:** The best improvements are seen with Qwen2.5-Coder-7B, where up to 4 code revisions adhere to feedback. However, undercorrection and failure cases increase as well, leading to more evenly distributed results across the categories. This indicates inconsistent and unreliable behavior, rather than stable improvement. Such inconsistent behavior limits the practical application of the fine-tuned models severely. Evaluation by Simulation requires a high degree of adherence to feedback with minimal deviation. This is important to ensure that the code revisions reflect the feedback, rather than inconsistencies of the model.

**Model size:** After supervised fine-tuning, the best performing model is Qwen2.5-Coder-7B. In contrast, Qwen2.5-Coder-32B does not show similar improvements and instead shifts towards undercorrection and failure. This suggests that increasing the model size is insufficient to yield an increase in adherence to feedback within the scope of our experimental setups. This might indicate that our experimental setups do not fully leverage the potential of increasingly higher model sizes.

**Tendency to deviate:** The influence of supervised fine-tuning on the tendencies of the models varies. Supervised fine-tuning does not reduce the tendency for undercorrection in Llama-3.2-3B, it adds a tendency for failure instead. For Qwen2.5-Coder-7B, we observe a reduction in the number of overcorrections and an increase in undercorrection and in adherence to feedback after training. This suggests that LLMs with a tendency for overcorrection are more responsive to fine-tuning, while a tendency for undercorrection in LLMs persists despite fine-tuning. Consequently, it appears that supervised fine-tuning

shifts tendencies along the following axis:

overcorrection  $\rightarrow$  adherence to feedback  $\rightarrow$  undercorrection  $\rightarrow$  failure

This might imply that a model with a tendency to overcorrect is better suited for supervised fine-tuning and a better starting point to fully unlock adherence to feedback.

**Failure rate:** Supervised fine-tuning introduces a higher number of failures across most experimental setups. Llama-3.2-3B is affected the most with up to 4 of the generated code revisions in the failure category. The models learn to generate code revisions by partially correcting errors and partially retaining errors. However, the changes in the code revisions are not fully grounded in the given feedback. The undercorrections and overcorrections that both are present in the code revision result in a placement in the failure category.

**Learning rate:** Across all configurations, we find no single optimal learning rate that results in improvement for adherence to feedback. Instead, we find that higher learning rates correlate with more failure cases, while lower learning rates correlate with an increase in undercorrection. Increases in adherence to feedback occur only for specific combinations of models, prompts, and learning rates. This suggests that adherence to feedback is not a robust capability of small-scale LLMs.

**Format Sensitivity:** Our results show that Qwen2.5-Coder-7B and Llama-3.2-3B are performing slightly better when prompted with markdown. Llama-3.2-3B-XML generates not a single code revision with adherence to feedback, while Llama-3.2-3B-markdown generates at most 2 cases of adherence. Qwen2.5-Coder-7B-XML and Qwen2.5-Coder-7B-markdown differ by an average of 0.66 cases in adherence to feedback. However, we see that the overall distributions remain relatively comparable within the same models. Neither XML nor markdown can consistently yield a high number of code revisions with adherence to feedback. This suggests that small-scale LLMs are marginally sensitive towards the format of the prompt. However, neither XML nor markdown fundamentally change the models adherence to feedback.

### 5.3. GRPO Training

We discuss the results of the experiments with GRPO fine-tuning. GRPO trains the LLMs by rating a batch of generated responses. This allows us to observe to what extent GRPO can adjust existing tendencies in a model. We utilize GRPO training to construct reasoning models. This allows us to answer how reasoning capabilities influence adherence to feedback.

**Stable impact:** GRPO fine-tuning has a stable and predictable influence on the performance of both configurations. For XML, Qwen2.5-Coder-7B-Instruct shows a stable fluctuation between overcorrection and failure cases. For markdown, we see a steady increase in overcorrection and decrease in failure cases, and an elimination of previous undercorrection. The adherence to feedback improves marginally for markdown, while no adherence to feedback occurs for XML. The persisting tendency towards overcorrection stands in misalignment with our training objective. This indicates a potential weakness in the underlying interaction between Qwen2.5-Coder-7B-Instruct and the reward functions, rather than a limitation of GRPO as a viable training technique to unlock adherence to feedback.

**Tendency to deviate:** Qwen2.5-Coder-7B-Instruct shows a tendency for overcorrection. For XML, this tendency is present prior to GRPO training and remains persistent throughout training. For markdown, we see a gradual shift towards overcorrection. This might indicate that the training objective aligns more with overcorrection than with adherence to feedback. The highest score of the reward functions is given by the judge with 4.0 points out of a total of 6.5 points. Therefore, the score of the judge determines the training objective. If the judge fails to reliably distinguish code revisions that adhere to feedback from code revisions that contain overcorrections, then this can result in the observed tendency for overcorrection.

**Format sensitivity:** Qwen2.5-Coder-7B-Instruct shows a marginally better performance after training with markdown, consistently generating at least 1 code revision that adheres to feedback. For XML, adherence to feedback remains at 0 cases throughout training. It appears that XML and markdown create different starting conditions for GRPO training. GRPO trains the LLM based on the generated responses. When the LLM frequently generates code revisions with overcorrections, an existing tendency towards overcorrection is amplified. This can result in an optimization towards a local optimum rooted in overcorrection. For XML, Qwen2.5-Coder-7B-Instruct starts and remains near this optimum. For markdown, Qwen2.5-Coder-7B-Instruct slowly converges towards this optimum. This highlights an underlying bias for overcorrection, that is strongly pronounced in XML.

**Reasoning model:** GRPO successfully trains Qwen2.5-Coder-7B-Instruct to express reasoning capabilities. After processing 1200 samples, we observe that the reward functions for reasoning tags, answer tags, and the quantity of tags consistently rate the generated responses with a relatively high score. Across these reward functions, XML achieves an average score of at least 0.46 points, while markdown achieves an average score of at least 0.475 points out of a total score of 0.5 points. This can be translated to a reliability of 92% for XML and 95% for markdown to generate responses with reasoning. In order to evaluate the impact of reasoning on adherence to feedback, observing a high reliability



is a crucial prerequisite. A high reliability validates that we are not just evaluating the ability to generate code revisions after reinforcement training. Instead, we can attribute the results to the acquired reasoning capabilities as well.

**Reasoning:** We enable Qwen2.5-Coder-7B-Instruct to generate responses that contain reasoning. However, we see no immediate impact on adherence to feedback after processing 1000 samples. Even after establishing reliable reasoning, adherence does not improve substantially after processing over 1200 samples. We conclude that Qwen2.5-Coder-7B-Instruct does not show substantial change in the ability to generate code revisions after acquiring reasoning capabilities. There are two possible explanations. First, the LLM generates reasoning that is irrelevant to the answer. Second, the LLM generates relevant reasoning, but the LLM is unable to translated the reasoning into enhanced adherence within the code revision. In both cases, we can identify a disconnect between reasoning and answer.

## 5.4. GRPO LLM-as-a-Judge

We discuss the results of the experiments of the LLM-as-a-judge approach in the context of GRPO training. The judge rates the generated responses with a score and steers the GRPO training. This allows us to answer to what extent large-scale LLMs can evaluate code revisions and determine adherence to feedback and deviation categories.

**Moderate accuracy:** We utilize GPT-4.1-mini with a concise prompt and appended code difference as the judge for GRPO training. The judge achieves a moderate accuracy of 88.84% on our evaluation data set. While this accuracy appears promising at first, our training results for Qwen2.5-Coder-7B-Instruct show that this accuracy does not translate into a substantial increase in adherence to feedback. This demonstrates that a moderate judge accuracy is not a sufficient metric that ensures effective GRPO training.

**Moderate specificity:** The judge achieves a moderate specificity of 88.48% on our evaluation data set. This would indicate a moderate detection of code revisions that do not adhere to feedback. However, this specificity is based on an imbalanced evaluation data set. Since the evaluation data set consists of 224 samples from the previous experiments with zero-shot prompting and supervised fine-tuning, the number of undercorrected cases is overrepresented with a total of 105 samples. Therefore, the reported specificity might not adequately represent the judge’s detection rate for overcorrection and failure cases.

**Imbalanced recall:** The judge achieves 90.91% recall on adherence to feedback and 92.38% recall on undercorrection. This suggests moderate detection for these categories. This matches partially with our results. Qwen2.5-Coder-7B-Instruct generates only up to

1 undercorrected code revision after GRPO training. However, a substantial increase in adherence to feedback is absent. This indicates that the initial adherence to feedback of Qwen2.5-Coder-7B-Instruct is insufficient to be amplified through GRPO training. The recall on the failure category is 23.26%. We see a decrease in the number of failure cases for the markdown prompt. This might indicate that the judge detects failure cases in practice, but misclassifies these failure cases as undercorrection cases instead. The recall for overcorrection is 55.81%. This appears to be the reason for the misalignment in the training objective. The judge seems to fail to consistently penalize overcorrections in code revisions. Consequently, Qwen2.5-Coder-7B-Instruct learns to optimize the score with code revisions that bypass the judge’s detection of overcorrections.

**Judge score:** Over the entire training duration, we observe a linear increase in the judge score for the code revisions generated by Qwen2.5-Coder-7B-Instruct. For XML, the final judge score reaches around 2.4 out of 4.0 points. However, we find that no code revisions with adherence to feedback are generated on our evaluation data set under XML. This implies that the judge score does not successfully capture the notion of adherence to feedback. Rather, the LLMs learn to satisfy the judge through an alternative criterion. Since Qwen2.5-Coder-7B-Instruct shows a tendency towards overcorrection, we believe this criterion to be the generation correct and valid code.

**Judge Prompt:** We prompt the judge with the program code, code revision and code difference. In addition, we specifically prompt the judge to highlight any undocumented changes with respect to the given the feedback. This prompt design ensures that the judge receives all information that is necessary to maximize sensitivity towards overcorrections. Given the low recall for overcorrection of 55.81% and the tendency for overcorrection of Qwen2.5-Coder-7B-Instruct after GRPO training, we conclude that our employed prompting strategy is insufficient to steer the judge towards reliable detection of overcorrected code revisions. Consequently, the judge is limited in the ability to reliably determine scores for adherence to feedback.

## 5.5. Implications

We answer the research question. Adherence to feedback appears as a severely limited capability of small-scale LLMs. The baseline performance demonstrates that we cannot take adherence to feedback for granted when deploying LLMs out-of-the-box. LLMs demonstrate tendencies towards overcorrection, undercorrection and failure that vary between different technologies. While certain techniques can influence adherence to feedback, no single technique achieves a substantial increase. We see that neither prompt format, nor model family, nor model size, nor fine-tuning, nor reasoning capabilities establish reliable

adherence to feedback. Instead, these techniques can slightly alter and redistribute the observed tendencies. In particular, supervised fine-tuning can unlock adherence to feedback to an inconsistent degree within model families. Consequently, we view adherence to feedback as an unreliable latent capability in certain models.

Supervised fine-tuning and GRPO training differ in their impact on the models, pose distinct challenges and serve different functions. Supervised fine-tuning yields the best results with 4 out of 8 code revisions adhering to feedback for specific configurations. We observe that supervised fine-tuning might be more suitable in combination with LLMs that show a tendency for overcorrections. Supervised fine-tuning impacts the models directly and encourages the acquisition of new behavior quickly, minimizing development costs. The training duration with supervised fine-tuning is shorter than with GRPO training, allowing for more configurations to be tested in the same time across multiple epochs and learning rates. The downside is that it is difficult to reason and interpret the results due to the black-box nature of the approach.

GRPO training achieves at most 2 out of 8 code revisions with adherence to feedback for the experimental setup with markdown. GRPO has a steady and predictable impact on the model, minimizing undercorrection and failure cases, but seems to primarily reinforce the tendency to overcorrect. This occurs due to a misalignment of the score provided by the judge in combination with an initial deficiency of the model to generate code revisions with adherence to feedback. Since we utilize an LLM-as-a-judge approach to score adherence to feedback, GRPO training sharply increases in complexity, duration and cost compared to supervised fine-tuning. Not only does the LLM generate a group of responses, but the judge evaluates each response by generating an additional response-score pair. This increase in complexity combined with the observed software crashes limits the scalability of the technique.

Both training techniques are viable to facilitate an increase in adherence to feedback, but neither technique can fully minimize the tendency to deviate at the same time. This results in unreliable and inconsistent behavior. Consequently, the impact of both training techniques is not very meaningful. We argue that small-scale LLMs are not suitable for practical application scenarios that require a high degree of adherence to feedback. For Evaluation by Simulation in particular, this means code revisions do not necessarily reflect the given code-feedback alone, but are impacted by the tendency for deviation. Therefore, small-scale LLMs do not make for reliable artificial students.

While the previous results for small-scale LLMs suggest a challenge in the automated generation of code revisions with adherence to feedback, the LLM-as-a-judge approach reveals that the automated evaluation poses a challenge as well. Despite our efforts to

increase the sensitivity of GPT-4.1-mini towards undocumented changes in the code revision, the detection of overcorrections remains low. This poses a problem during GRPO training because low detection rates limit the potential of the large-scale LLM to instill adherence to feedback in the small-scale LLM. For example, valid and correct code revisions can bypass the detection of the judge. When evaluating a judge model, it is crucial to consider the recall of multiple categories rather than relying on accuracy alone.

## 6. Related Work

Our work intersects with a range of open research topics. These topics include program synthesis, program repair, and instruction-following. We review adherence to feedback as a blend of these topics. To the best of our knowledge, adherence to code-feedback is not widely studied. Adherence to feedback does not fit into program repair alone, but rather studies the behavior of LLMs when prompted for program repair with additional constraints.

Code generation or program synthesis deals with the construction of program code from a given specification. The capability of LLMs to generate program code from natural language descriptions is measured through large scale tests using benchmarks such as APPS [17], HumanEval [6], and MBPP [2]. These benchmarks prompt the LLM to implement a programming task from scratch and validate the program code by running unit tests. APPS consists of 10,000 python exercises, HumanEval consists of 164 python exercises, and MBPP consists of 974 python exercises. However, these benchmarks are limited to measuring the functional correctness of the program code. The problem with static benchmarks is that these benchmarks can appear partially or in full in the training data set of newer LLMs. LiveCodeBench aims to periodically gather new tasks to address this problem [23].

Reasoning models deploy chain-of-thought [50] and multi-step thinking and can achieve higher performance than earlier methods for code generation. DeepSeek-R1 [16] achieves an average score of 71.2% and performs 7.4% better than ChatGPT-4 [1] on code generation tasks. LLMs perform worse on more complex tasks, with DeepSeek-R1 achieving a score of 55.0% on advanced programming problems [11]. While this highlights substantial coding capabilities, LLMs are not without flaws and still frequently fail to generate program code correctly.

There are different types of errors, such as syntax errors, runtime errors, and functional errors. Syntax errors describe errors that can be identified by a compiler. Runtime errors occur during execution of the program code. Functional errors refer to problems that do not satisfy the specification. For DeepSeek-R1, the error distribution is 0.0% syntax errors, 30.6% runtime errors, and 69.4% functional errors. The tendency to generate program code that looks correct but does not always run is consistent across multiple tested models [11]. For our work, it is relevant to note that the major source of errors in generated code originates from a misalignment with the task specification, and not from an inability to generate correct and valid code. This seems to be a systemic problem across all LLMs.

Additional work on CodeGen [38], PanGu-Coder [8], and Codex [6] highlights that misinterpretation of the task (20.77%), additional non-prompted consideration (8.15%), and redundancies (6.11%) represent a substantial portion of errors. Incomplete implementation of code constitutes 9.57% of errors. Syntax errors occur in 6.11% of cases, while runtime errors occur in 39.3% of generated code [47]. These results show that LLMs express different types of deviation when failing to adhere to the task.

Automated program repair deals with the identification and correction of errors in existing program code. SWE-Fixer is an LLM-based approach to automatically resolve issues on GitHub by extracting and implementing feedback provided by the users on these issues. SWE-Fixer uses Qwen2.5-7B to identify and gather the necessary files and uses Qwen2.5-72B to implement the code correction through chain-of-thought. Qwen2.5-72B is further fine-tuned on over 110 thousand samples. SWE-Fixer achieves a performance of 23.3% on the SWE-Bench Lite and 30.2% on the SWE-Bench Verified [52]. This is relevant for our work, since SWE-Bench utilizes a model from the Qwen2.5 model family in a practical application. Despite deploying a larger 72B parameter model, SWE-Fixer allows us to draw similar conclusions. While LLMs can successfully generate code corrections, these code corrections are infrequent and unreliable.

GPT-4 is capable of implementing simple errors into otherwise correct program code. Across 375 generated code revisions, GPT-4 does not generate erroneous code in only 12 cases (3.2%). When prompted for specific types of error, GPT4 follows the instruction in 87% of cases. In 8% of cases, GPT-4 generates other types of errors that have not been requested in the prompt [27]. This is relevant to our study, since it shows that LLMs can be utilized in the generation of synthetic data sets containing erroneous code.

Instruction-following is closely related to adherence to code-feedback and describes the capability of LLMs to adhere to instructions given in natural language in a general context. The evaluation of instruction-following can be measured with IFEval. IFEval is a benchmark consisting of 541 prompts with one or more instructions that are phrased in different variations. The idea is to measure the performance of LLMs on automatically verifiable instructions. This includes checks for keywords, text length, and the response format [55]. GPT-4 achieves a score of 85.37% on IFEval [55] and Qwen2.5-Coder-7B-Instruct achieves a score of 58.6% [21].

## 7. Conclusion

Within the scope of this master’s thesis, we set out to investigate the capability of small-scale LLMs for adherence to feedback. We prompted LLMs to implement code-feedback into program code and primarily evaluated the generated code revisions for overcorrection, undercorrection, and for severe failure to produce valid responses. Our work shows that adherence to feedback poses a substantial challenge for small-scale LLMs. We have found 2 out of 35 configurations that achieve the best performance by generating 4 out of 8 code revisions with adherence to feedback. We conclude that adherence to feedback is present in certain small-scale LLMs, but it remains an inconsistent and unreliable capability.

Across our experimental setups, we saw that small-scale LLMs show a tendencies to deviate towards a specific category. For 5 of 35 configurations, we identify a tendency for overcorrection. For 17 of 35 configurations, we identify a tendency for undercorrection. For 5 of 35 configurations, we identify a tendency for failure. We observed that our experiments with multiple prompt formats, model families, model sizes, fine-tuning techniques, and with reasoning capabilities primarily alter, redistribute, and even reinforce these tendencies, rather than strengthening adherence to feedback.

Supervised fine-tuning is a simple and direct training technique and contributed to the highest increase in adherence to feedback within our experimental setups. GRPO is a more complex training technique and introduces more points of failure. We saw that GRPO training in combination with LLM-as-a-judge can lead to stagnant improvements. Despite specific prompting to increase the sensitivity for overcorrections, the LLM-as-a-judge approach achieves a recall of 55.81%. Automated generation and automated evaluation are not robust enough for further reinforcement during GRPO training. This makes supervised fine-tuning a more accessible technique to unlock adherence to feedback.

We have observed that small-scale LLMs can implement code-feedback into program code, but small-scale LLMs frequently fail to adhere to feedback. This poses a substantial problem for practical applications, where adherence to feedback is a major constraint, and makes small-scale LLMs unsuitable as artificial students for Evaluation by Simulation in particular. Generated code revisions do not solely reflect the provided code-feedback. Instead, generated code revisions are severely impacted by existing tendencies in the model. Consequently, Evaluation by Simulation is unable to reliably assess the quality of the code-feedback through automated evaluation of the generated code revision.

This work establishes adherence to feedback as an important and valuable, yet overlooked, capability of LLMs. The current development of LLMs relies too heavily on measuring the correctness of the generated program code to report on coding capabilities. We argue that this training objective is too narrow, because it can unintentionally limit the

---

versatility of LLMs in practical applications, where correctness is not the primary objective. Correctness should not stand in conflict with adherence to feedback. This requires advanced tools and distinct evaluation metrics to enhance these capabilities. We hope that our work provides an opportunity to rethink current design principles to account for adherence to feedback.



## 8. Limitations

In our work, we primarily focus on adherence to feedback in the context of partially correcting erroneous program code. The feedback in our evaluation data set prompts the LLM to implement code corrections into the program code and does not mention all errors. Our results might not be transferable to tasks where the feedback addresses all errors in the program code, because the occurrence of overcorrections would be eliminated. It is possible that adherence to feedback would be substantially higher in that setting.

We have claimed that the automated evaluation for adherence to feedback poses a challenge. However, in the setting where we eliminate overcorrections by targeting all errors in the program code, we could perform automated tests on the generated code revisions. For example, we could run the prompts and unit tests from the MBPP evaluation data set on the generated code revisions. This would be the de facto industry standard to measure coding capabilities. While this is not entirely in line with our definition of adherence to feedback, we could more reliably measure the undercorrection and failure cases in this way. We argue that this does not put the LLMs under the same testing conditions, since LLMs are already pre-trained for that exact setting, where the LLMs have to generate purely correct and valid program code. We want to observe the behavior when the generation of overcorrections is a tempting possibility for the LLMs. In addition, we still would need to evaluate the tendency for overcorrections by hand anyway.

While our results for Llama-3.2-3B indicate low adherence to feedback, we cannot generalize this statement to all 3B parameter models. This is because Llama-3.2-3B is not necessarily optimized for our coding requirements. Llama-3.2-3B excels at dialog, text summarization, and as writing assistance. There is a possibility that other 3B parameter models show better adherence to feedback and respond more positively to fine-tuning.

We deploy QLoRA to load and train the LLMs. Since QLoRA uses a special NF4 datatype with 4-bit, the parameters of the LLMs are loaded in lower precision. This might negatively impact the LLMs, because the calculations throughout the layers of the LLMs amplify the quantization error. As a result, we might see a degradation of the quality of the generated responses. We could see different results for adherence to feedback, if we consider loading and training the LLMs with a higher precision datatype, but this would increase the GPU memory requirements.

Throughout our experiments, we utilize German to prompt the LLMs. This could have an important influence on our results. While Qwen2.5-Coder-7B might be able to process German text, it is not necessarily specialized for coding exercises based on German instructions. Since Qwen2.5-Coder-7B is pre-trained on a variety of coding projects and natural language documents available on the world wide web, we can assume that English

is the dominant language of the model. Therefore, prompting in English might result in better adherence to feedback. The language barrier is further amplified because our prompts represent a mixed document type. LLMs must understand the task description, identify the feedback, and implement the feedback into the given program code. The interpretation of our prompts might be hindered if the model is not trained to handle this mixed document type with German instructions specifically.

Our training data set has a moderate size with 2 million tokens across 5000 samples. Naturally, this is not comparable to the 5.5 trillion tokens used in the pre-training of the models. We cannot expect our training to fundamentally change the behavior of the models or teach a completely new capability from scratch. However, our data set size is suitable to steer the models and adjust the existing behavior.

The effectiveness of supervised fine-tuning depends on the quality of the utilized training data set. Our training data set is based on unfiltered interactions between real world students and an artificial teacher. These interactions can result in noise. Since the students were not obligated to implement the provided feedback, the training data set can contain samples where no changes are made to the program code. This can negatively impact our attempts to train adherence to feedback because the model might learn that ignoring the feedback is a valid option.

The scope of the GRPO training in our experiments is limited to a single learning rate, 2 prompt scenarios, and 6 responses per processed sample. An important aspect of reinforcement learning is the exploration-exploitation paradigm. A higher number of generated responses could potentially allow the model to explore and discover more code revisions that adhere to feedback. On the other hand, when the model fails to generate a single code revision with adherence to feedback for a group of responses, then GRPO is unable to optimize the model. This indirectly influences how many samples can effectively contribute to the training. Furthermore, additional prompting scenarios could offer more starting points to steer the model into different directions.

GRPO training successfully achieves reasoning models that show reliable reasoning capabilities after processing 1200 samples. However, our experiments do not evaluate the quality of the generated reasoning. This leaves us without a direct explanation why reasoning capabilities do not improve adherence to feedback. There is the possibility that the generated reasoning has the opposite effect. For example, the model could generate reasoning that suggests additional corrections that fall outside of the feedback. Our setup does not specifically attempt to strengthen the connection between reasoning and adherence to feedback. Future work could evaluate the generated reasoning with an LLM-as-a-judge to determine whether the reasoning is aligned with adherence to feedback.

The evaluation of our experimental setups utilizes a small data set of 8 exercises. Therefore, our results should not be used to generalize the performance claims of individual configurations. Rather, our results capture consistent behavior that can be observed across multiple experiments. This behavior can be used to characterize the tendencies of small-scale LLMs in a more general sense.

Adherence to feedback might be a necessary condition, but not a sufficient condition for Evaluation by Simulation. Assuming that we would have observed a high degree of adherence to feedback, that would not immediately imply that Evaluation by Simulation is feasible. For example, an LLM could generate an identical code revision for two separate instances of feedback, even when one instance of feedback is clearly of inferior quality. Despite fully established adherence to feedback, the quality of the code revision would not reflect the quality of the feedback. The outlined example would require more detailed measurements and experiments.

## 9. Future Work

We suggest that future work studies the connection between language and adherence to feedback. This could be achieved by repeating our experiments but with prompts in English. This could help align the models more closely to the task. Success would inform research to invest into models that are specifically tuned to the same language as the prompts and would reveal a language-specific property of adhere to feedback.

A major problem that we encountered was the tendency to overcorrect. The ability of the model to predict the next token might be limited to mostly predicting correct and valid code. Future work could intentionally incorporate erroneous code into the training data set to change that behavior of the model. This could help the model to generate and reproduce existing errors, rather than treating all errors as undesirable.

The size of the model did not appear to be a deciding factor in our experiments. Should small-scale LLMs prove to be unsuitable for adherence to feedback, then further work on large-scale LLMs could be necessary. This could explore whether adherence to feedback is an emergent capability that is difficult to reproduce with small-scale LLMs.

Zero-shot prompting was not sufficient to steer the LLMs towards adherence to feedback. Future work could evaluate the impact of more elaborate prompting strategies. Each prompt could be specifically tailored to the individual model, rather than using a single prompt for all models. Ideally, prompt engineering alone would be sufficient, because that would avoid the need for more costly techniques such as fine-tuning.

GRPO training aimed to establish reasoning capabilities and to strengthen adherence to feedback simultaneously. This might have resulted in an inefficient utilization of our training data set. We have observed that reliable reasoning required around 1200 samples to be processed. These initial 1200 samples might not have effectively contributed to the training of adherence to feedback. We suggest that future work considers to repeat these initial 1200 samples or to perform at least 2 epochs to maximize the training data set utilization.

We employed an LLM-as-a-judge approach to determine the score of the generated code revisions. A single judge in combination with a single prompt, that attempts to cover every category, might be inadequate. Future work could evaluate whether an ensemble of judges, for example one judge per category, can work together to yield an increase in recall. In particular, this could help to improve the detection of overcorrections by optimizing the judge for that category alone.

LLM-as-a-judge is a slow and expensive evaluation technique. There are two trivial cases that could be evaluated with simpler algorithms. First, if the generated code revision is

an exact copy of the initial program code, then this could be detected by parsing both documents into abstract syntax trees. If both abstract syntax trees are identical, then the code revision is undercorrected. Second, if the generated code revision corrects all errors, then this could be detected by running the code revision through a compiler. This could help to avoid prompting the judge unnecessarily in these two trivial cases.

Our final suggestion is to combine both GRPO training and supervised fine-tuning within a unified training approach. Throughout all of our experiments, we have observed that supervised fine-tuning does not increase tendencies towards overcorrection, while GRPO training does not increase tendencies towards undercorrection. For certain configurations, we even saw that either training technique reduces their respective tendency instead. Future work could investigate this complementary behavior by measuring the effect of applying and alternating between both training techniques. This could have the potential to achieve more reliable adherence to feedback in small-scale LLMs.

# Acknowledgment

We thank Sven Jacobs for supervising this master's thesis and providing guidance and access to the necessary hardware for running and fine-tuning large language models. The Chair of Operating Systems and Distributed Systems at the University of Siegen provided the hardware for this research and generously covered the accumulated costs.

## References

- [1] Josh Achiam et al. “Gpt-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [2] Jacob Austin et al. “Program synthesis with large language models”. In: *arXiv preprint arXiv:2108.07732* (2021).
- [3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural machine translation by jointly learning to align and translate”. In: *arXiv preprint arXiv:1409.0473* (2014).
- [4] Leonardo Berti, Flavio Giorgi, and Gjergji Kasneci. *Emergent Abilities in Large Language Models: A Survey*. 2025. arXiv: 2503.05788 [cs.LG]. URL: <https://arxiv.org/abs/2503.05788>.
- [5] Yupeng Chang et al. “A Survey on Evaluation of Large Language Models”. In: *ACM Trans. Intell. Syst. Technol.* 15.3 (Mar. 2024). ISSN: 2157-6904. DOI: 10.1145/3641289. URL: <https://doi.org/10.1145/3641289>.
- [6] Mark Chen et al. “Evaluating large language models trained on code”. In: *arXiv preprint arXiv:2107.03374* (2021).
- [7] Kyunghyun Cho et al. “Learning phrase representations using RNN encoder–decoder for statistical machine translation”. In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 2014, pp. 1724–1734.
- [8] Fenia Christopoulou et al. “Pangu-coder: Program synthesis with function-level language modeling”. In: *arXiv preprint arXiv:2207.11280* (2022).
- [9] Michael Han Daniel Han and Unsloth team. *Unsloth*. 2023. URL: <http://github.com/unslothai/unsloth>.
- [10] Tim Dettmers et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. arXiv: 2305.14314 [cs.LG]. URL: <https://arxiv.org/abs/2305.14314>.
- [11] Shihan Dou et al. “What is wrong with your code generated by large language models? An extensive study”. In: *Science China Information Sciences* 69.1 (2026), p. 112107.
- [12] Iria Estévez-Ayres et al. “Evaluation of LLM Tools for Feedback Generation in a Course on Concurrent Programming”. In: *International Journal of Artificial Intelligence in Education* 35.2 (June 2025), pp. 774–790. ISSN: 1560-4306. DOI: 10.1007/s40593-024-00406-0. URL: <https://doi.org/10.1007/s40593-024-00406-0>.

- [13] Hagit Gabbay and Anat Cohen. “Combining LLM-generated and test-based feedback in a MOOC for programming”. In: *Proceedings of the Eleventh ACM Conference on Learning@ Scale*. 2024, pp. 177–187.
- [14] Majid Ghasemi and Dariush Ebrahimi. “Introduction to reinforcement learning”. In: *arXiv preprint arXiv:2408.07712* (2024).
- [15] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: <https://arxiv.org/abs/2407.21783>.
- [16] Daya Guo et al. “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning”. In: *Nature* 645.8081 (2025), pp. 633–638.
- [17] Dan Hendrycks et al. “Measuring coding challenge competence with apps”. In: *arXiv preprint arXiv:2105.09938* (2021).
- [18] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural computation* 9.8 (1997), pp. 1735–1780.
- [19] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. arXiv: 2106.09685 [cs.CL]. URL: <https://arxiv.org/abs/2106.09685>.
- [20] Huggingface. *Huggingface Models*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/models>.
- [21] Binyuan Hui et al. “Qwen2.5-Coder Technical Report”. In: *arXiv preprint arXiv:2409.12186* (2024).
- [22] Sven Jacobs and Steffen Jaschke. “Evaluating the Application of Large Language Models to Generate Feedback in Programming Education”. In: *2024 IEEE Global Engineering Education Conference (EDUCON)*. 2024, pp. 1–5. DOI: 10.1109/EDUCON60312.2024.10578838.
- [23] Naman Jain et al. “Livecodebench: Holistic and contamination free evaluation of large language models for code”. In: *arXiv preprint arXiv:2403.07974* (2024).
- [24] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- [25] Taku Kudo and John Richardson. “SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing”. In: *Proceedings of the 2018 conference on empirical methods in natural language processing: System demonstrations*. 2018, pp. 66–71.
- [26] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In: *Advances in neural information processing systems* 33 (2020), pp. 9459–9474.



- [27] Stephen MacNeil et al. “Synthetic students: A comparative study of bug distribution between large language models and computing students”. In: *Proceedings of the 2024 on ACM Virtual Global Computing Education Conference V. 1*. 2024, pp. 137–143.
- [28] Meta. *Llama 3.2: Revolutionizing edge AI and vision with open, customizable models*. Accessed: 2025-09-24. 2024. URL: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>.
- [29] Meta. *Llama-3.1-405B*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/meta-llama/Llama-3.1-405B>.
- [30] Meta. *Llama-3.1-70B*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/meta-llama/Llama-3.1-70B>.
- [31] Meta. *Llama-3.1-8B*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/meta-llama/Llama-3.1-8B>.
- [32] Meta. *Llama-3.2-1B*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/meta-llama/Llama-3.2-1B>.
- [33] Meta. *Llama-3.2-3B*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/meta-llama/Llama-3.2-3B>.
- [34] Tomas Mikolov et al. “Efficient estimation of word representations in vector space”. In: *arXiv preprint arXiv:1301.3781* (2013).
- [35] Osval Antonio Montesinos López, Abelardo Montesinos López, and Jose Crossa. “Fundamentals of artificial neural networks and deep learning”. In: *Multivariate statistical machine learning methods for genomic prediction*. Springer, 2022, pp. 379–425.
- [36] Ha Nguyen and Vicki Allan. “Using GPT-4 to provide tiered, formative code feedback”. In: *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 2024, pp. 958–964.
- [37] Minh Nhat Nguyen et al. “Turning up the heat: Min-p sampling for creative and coherent llm outputs”. In: *arXiv preprint arXiv:2407.01082* (2024).
- [38] Erik Nijkamp et al. “Codegen: An open large language model for code with multi-turn program synthesis”. In: *arXiv preprint arXiv:2203.13474* (2022).
- [39] NVIDIA. *CUDA C++ Programming Guide*. Accessed: 2025-11-8. 2025. URL: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [40] Qwen. *Qwen2.5-Coder-32B*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/Qwen/Qwen2.5-Coder-32B>.
- [41] Qwen. *Qwen2.5-Coder-7B*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/Qwen/Qwen2.5-Coder-7B>.

- [42] Qwen. *Qwen2.5-Coder-7B-Instruct*. Accessed: 2025-09-24. 2024. URL: <https://huggingface.co/Qwen/Qwen2.5-Coder-7B-instruct>.
- [43] Sander Schulhoff et al. “The prompt report: A systematic survey of prompt engineering techniques”. In: *arXiv preprint arXiv:2406.06608* (2024).
- [44] Zhihong Shao et al. “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).
- [45] Ilya Sutskever, James Martens, and Geoffrey E Hinton. “Generating text with recurrent neural networks”. In: *Proceedings of the 28th international conference on machine learning (ICML-11)*. 2011, pp. 1017–1024.
- [46] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. “Sequence to sequence learning with neural networks”. In: *Advances in neural information processing systems* 27 (2014).
- [47] Florian Tambon et al. “Bugs in large language models generated code: An empirical study”. In: *Empirical Software Engineering* 30.3 (2025), p. 65.
- [48] Gemini Team et al. “Gemini: a family of highly capable multimodal models”. In: *arXiv preprint arXiv:2312.11805* (2023).
- [49] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [50] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.
- [51] Thomas Wolf et al. “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. URL: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
- [52] Chengxing Xie et al. “Swe-fixer: Training open-source llms for effective and efficient github issue resolution”. In: *Findings of the Association for Computational Linguistics: ACL 2025*. 2025, pp. 1123–1139.
- [53] An Yang et al. “Qwen2 Technical Report”. In: *arXiv preprint arXiv:2407.10671* (2024).
- [54] Biao Zhang et al. “When scaling meets llm finetuning: The effect of data, model and finetuning method”. In: *arXiv preprint arXiv:2402.17193* (2024).
- [55] Jeffrey Zhou et al. “Instruction-following evaluation for large language models”. In: *arXiv preprint arXiv:2311.07911* (2023).

## A. Appendix

In chapter 3.2, our prompts for zero-shot prompting are listed in English. Below, we list the German equivalents that we utilize in our experiments:

### Prompt 1: Markdown

Du bist ein Neuling und Anfänger in Programmieren. Hier ist die Aufgabe.  
Schreibe eine Revision des inkorrekten Programms. Das inkorrekte Programm hat syntaktische und semantische Fehler und ist unvollständig.

Das inkorrekte Programm sieht so aus.

```
```python
    [ program code ]
```
```

Das Feedback ist wie folgt. [ feedback ]

Das Ziel der Revision ist die Probleme des inkorrekten Programms zu behalten.  
Die Revision des Programms ist absichtlich fehlerhaft und setzt nur das Feedback um. Vermeide Kommentare und Erklärungen in der Revision.

Revision des Programms:

```
```python
```

Prompt 2: XML

Du bist ein Experte für Software Fehler und Mängel.

Du demonstrierst typische Flüchtigkeitsfehler in `<programm>`.

Für die Präsentation hast du den folgenden `<programm>` Code geschrieben.

```
<programm>
    [ program code ]
</programm>
```

Du fragst das Publikum nach `<feedback>`.

```
<feedback>
    [ feedback ]
</feedback>
```

Du implementierst den `<code>` genau so, wie es in `<feedback>` gegeben wurde, und ausschließlich wie es in `<feedback>` gegeben wurde.

Alle sonstigen Fehler werden unverändert in `<code>` beibehalten, um weiteres `<feedback>` vom Publikum zu erhalten.

```
<code>
```

Eigenständigkeitserklärung

Ich versichere, dass ich die schriftliche Ausarbeitung selbständig angefertigt und keine anderen als die angegebenen Hilfsmittel benutzt habe. Alle Stellen, die dem Wortlaut oder dem Sinn nach (inkl. Übersetzungen) anderen Werken entnommen sind, habe ich in jedem einzelnen Fall unter genauer Angabe der Quelle (einschließlich des World Wide Web sowie generativer KI und anderer elektronischer Datensammlungen) deutlich als Entlehnung kenntlich gemacht. Dies gilt auch für angefügte Zeichnungen, bildliche Darstellungen, Skizzen und dergleichen. Ich nehme zur Kenntnis, dass die nachgewiesene Unterlassung der Herkunftsangabe als versuchte Täuschung gewertet wird.

Ich bestätige, dass die elektronische Version inhaltlich mit der gedruckten Version übereinstimmt.

Datum

Unterschrift

Declaration of Authorship

I certify that I have prepared this written work independently and have not used any aids other than those specified. I have clearly identified all passages that are taken from other works in terms of wording or meaning (including translations) as borrowed material in each individual case, stating the exact source (including the World Wide Web as well as generative AI and other electronic data collections). This also applies to attached drawings, pictorial representations, sketches and the like. I acknowledge that the proven omission of the indication of origin will be regarded as attempted deception.

I confirm that the content of the electronic version is the same as the printed version.